

PaperPass[免费版]查重报告

简明打印版

查重结果(相似度):

总体: 18%

本地库: 18% (本地库包含学术联合库、期刊库、学位库、会议库、共享联合库)

互联网: (免费版不检测互联网资源)

检测版本: 免费版(仅检测中文)

报告编号: 2VD369FCBD4CAE973

论文题目: 基于时序预测的门诊挂号与电子病历一体化管理系统的设计与实现

论文作者: 佚名

论文字数: 29650

段落个数: 1153

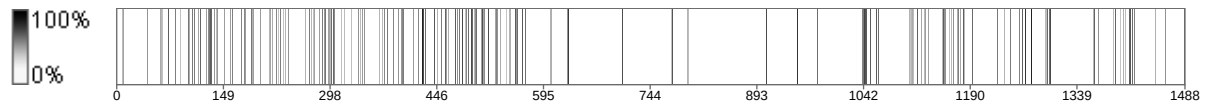
句子个数: 1488

提交时间: 2026-5-8 0:26:52

比对范围: 学术联合库、期刊库、硕博学位库、会议库、共享联合库

查询真伪: <https://www.paperpass.com/check>

句子相似度分布图:



本地库相似资源列表(学术联合库、期刊库、硕博学位库、会议库、共享联合库):

1. 相似度: 6.6% 来源: 学术联合库

查重 41%

基于时序预测的门诊挂号与电子病历一体化管理系统的 设计与实现

摘要：随着医疗信息化的不断发展，传统门诊挂号系统与电子病历系统多为独立运行，存在数据共享不足、流程割裂及资源配置不合理等问题。针对上述不足，本文设计并实现了一种基于时序预测的门诊挂号与电子病历一体化管理系统。系统采用前后端分离架构，基于 Vue 与 Spring Boot 技术实现，完成了挂号管理、医生排班、电子病历及权限管理等功能模块，实现了门诊业务数据的统一管理与协同处理。在此基础上，引入时间序列分析方法构建挂号量预测模型，对历史数据进行建模分析，实现对未来门诊需求的预测，并提供辅助挂号推荐服务，从而提升医疗资源配置的合理性。通过系统设计与实现，并结合功能测试与性能测试对系统进行验证。结果表明，该系统能够有效整合门诊挂号与电子病历信息，提高业务处理效率，并在一定程度上优化患者就诊流程。研究表明，将信息管理系统与数据预测方法相结合，有助于提升门诊服务的智能化水平，具有一定的应用价值。

查重 43%

查重 51%

关键词：门诊挂号系统；电子病历；前后端分离架构；时序预测模型；ARIMA 模型

Design and Implementation of an Integrated Outpatient Registration and Electronic Medical Record Management System Based on Time Series Forecasting

Abstract: With the development of medical informatization, traditional outpatient registration systems and electronic medical record systems are often developed and operated independently, resulting in insufficient data sharing, fragmented processes, and inefficient resource allocation. To address these problems, this paper designs and implements an integrated outpatient registration and electronic medical record management system based on time series forecasting. The system adopts a front-end and back-end separated architecture and is developed using Vue and Spring Boot. It integrates multiple functional modules, including outpatient registration management, doctor scheduling, electronic medical records, and user permission management, enabling unified management and collaborative processing of outpatient data. On this basis, a time series analysis method is introduced to build a registration volume prediction model. By analyzing historical data, the system can predict future outpatient demand and provide auxiliary registration recommendations, thereby improving the rational allocation of medical resources. Through system implementation and testing, including functional and performance evaluations, the effectiveness of the system is verified. The results show that the system can effectively integrate outpatient registration and electronic medical record data, improve operational efficiency, and optimize the patient visit process to a certain extent. This study demonstrates that combining information management systems with data-driven forecasting methods can enhance the intelligence level of outpatient services and has practical application value.

Key words: outpatient registration system; electronic medical record; Frontend-backend separation architecture; time series forecasting; ARIMA model

(目录)

第一章 绪论

1.1 研究背景及意义

1.1.1 研究背景

近年来，信息技术在医疗领域的渗透速度明显加快，门诊服务模式也在悄然发生变化。传统以人工为主的管理方式，在早期尚能支撑基本运转，但随着门诊量持续增长，这种模式逐渐暴露出效率瓶颈。以门诊环节为例，从挂号、候诊到病历记录，多个流程仍依赖人工操作^{[9][10]}，**不仅处理效率受限，还容易因人为因素导致信息遗漏或错误。**尤其是在大型医院，门诊医生日均接诊量通常达到数十甚至上百人次，信息处理负担较重。在此情况下，单纯依赖人工管理方式已难以保证系统长期稳定运行。

与此同时，门诊流程复杂性对患者就医体验也产生直接影响。排队时间较长、流程透明度不足以及重复操作频繁等问题，在就诊高峰时段尤为突出。相关研究表明，分时段预约能够显著降低候诊时间，但在实际运行中，由于缺乏有效的数据支撑与动态调度机制，其实施效果仍存在不稳定性^[13]。因此，单一的信息化手段虽可缓解部分门诊压力，但尚未形成系统化的优化能力。

在政策层面，国家近年来持续推动“智慧医院”建设，强调以电子病历为核心，推动医疗服务、管理与资源配置的协同发展。这一政策导向促使医院信息系统由单一业务支持向一体化、数据驱动方向转变。然而，从实际应用情况来看，当前多数医院信息系统仍主要停留在功能整合阶段，对医疗数据的深度挖掘与分析利用仍较为有限，尤其在门诊场景中，基于历史数据进行预测与决策支持的应用尚未得到广泛推广^[10]。

综上所述，当前门诊系统在协同能力与预测能力方面仍存在一定不足。尽管现有系统功能较为完善，但在数据协同与智能预测方面仍有较大提升空间。若能够在现有预约挂号与电子病历系统基础上，引入时间序列分析方法，对门诊量变化趋势进行预测，并进一步指导排班与资源配置，则系统功能将从单纯的流程优化扩展至辅助医院运营决策层面，从而提升门诊管理系统的整体应用价值^[9]。

1.1.2 研究意义

围绕门诊管理系统的优化问题，本文的研究意义主要体现在三个层面。

从患者体验出发，信息化手段带来的改变更为直观。在线预约、分时就诊以及提醒机

制，使患者能够提前规划就诊时间，避免无效等待。^{查重 41%}相比传统模式，这种方式减少了就医过程中的不确定性，在一定程度上改善了整体就诊体验。

从管理与决策层面来看，引入数据分析方法具有更深层的意义。通过对历史门诊数据进行建模，例如采用 ARIMA 等时间序列模型，可以对未来门诊量变化趋势进行预测。已有研究表明，在季节性特征明显的医疗数据中，该类模型具有较好的拟合效果。基于预测结果进一步优化医生排班，不仅能够提升资源利用率，也为医院应对突发高峰提供参考依据。

从系统发展角度来看，本研究在传统门诊管理系统基础上引入数据分析能力，使系统由“被动记录工具”向“主动决策辅助工具”转变。这种转变符合当前智慧医疗系统的发展趋势，对于提升医疗信息系统智能化水平具有一定的理论意义与应用价值。

1.2 国内外研究现状

^{查重 72%}在国内，医疗信息化建设起步虽晚，但发展速度较快。近年来，“互联网+医疗”与智慧医院建设不断推进，多数大型医院已具备预约挂号、电子病历及排班管理等基础功能^[7]。从应用层面来看，这些系统在缓解排队压力、优化流程方面取得了一定成效。例如，通过移动端预约与分时段就诊，患者候诊时间明显缩短，门诊秩序得到改善^[11]。

然而，从更深入的角度分析，当前国内门诊系统仍存在一定共性问题。一是系统之间的协同程度有限，电子病历、挂号系统与排班模块往往相对独立，数据共享不足；二是功能侧重于流程支持，对数据的深度挖掘较少。例如，在排班安排中，多依赖经验判断，而非基于历史数据进行量化分析。已有研究指出，门诊预约模式（如提前预约、开放预约及混合模式）会直接影响资源利用效率，但相关研究更多停留在模型分析层面，尚未广泛融入实际系统中^[10]。

^{查重 62%}相比之下，国外在医疗信息系统方面起步较早，整体发展更为成熟。早在上世纪中后期，部分国家已开始探索医院信息系统的应用，并逐步实现从财务管理向临床业务扩展。进入信息化深度发展阶段后，国外研究更加注重系统集成与智能化应用。例如，通过将临床设备直接接入电子病历系统，实现检查数据的自动采集与同步更新，从而减少人工录入环节，提高数据准确性^{[5][14]}。

在数据分析方面，国外研究也走在前列。一些研究尝试将时间序列模型与外部因素（如气候、节假日）结合，对门诊量进行预测，并取得较高精度^[12]。这类方法不仅用于趋势分析，还进一步应用于资源调度与排班优化中，使系统具备一定的决策支持能力。

查重 42%

将国内外研究进行对比可以发现，两者的关注重点存在差异：国外更强调系统集成与智能分析，而国内则侧重于业务流程优化与系统建设本身。尤其是在数据驱动决策方面，国内相关研究仍有较大提升空间。基于上述现状，本文在系统设计过程中引入时间序列预测方法，以提升传统门诊系统的智能化水平。

1.3 研究内容与目标

1.3.1 论文的主要研究内容

查重 41%

本文围绕门诊管理信息系统展开，从系统实现与数据分析两个层面进行研究。

在系统层面，通过需求分析明确患者、医生及管理的功能需求，并在此基础上设计整体架构。系统采用前后端分离模式，实现用户管理、预约挂号、排班管理以及电子病历等核心功能，使各业务模块形成相对完整的闭环。

在实现过程中，选用主流 Web 开发技术完成系统构建，并结合认证机制与缓存技术，提高系统的安全性及响应效率。数据库设计方面，通过合理的实体关系建模，确保数据结构清晰、可扩展。

查重 64%

在此基础上，进一步引入数据分析模块。针对历史挂号数据，采用时间序列方法进行建模，对未来门诊量进行预测。结合预测结果，设计简单的排班优化策略，用于辅助管理人员进行资源配置。

查重 48%

总体而言，本文不仅实现门诊管理系统的基础功能，还在传统系统基础上引入数据驱动分析能力，使系统具备一定的预测与辅助决策功能。

1.3.2 系统的设计目标

查重 63%

本系统的设计目标主要包括以下三个方面。

查重 59%

其一，实现门诊业务流程的信息化管理。通过整合挂号、排班与电子病历等功能，减少人工操作环节，使业务流程更加顺畅。

查重 63%

其二，提升系统的可用性与安全性。在保证功能完整的前提下，通过合理的认证机制与数据保护措施，确保系统运行稳定可靠。

其三，引入数据分析能力，对门诊量变化进行预测，并基于预测结果提供排班参考方案。虽然预测结果无法完全替代人工决策，但可以作为辅助依据，提高资源配置的合理性。

1.4 技术路线

查重 59%
本研究的实施过程大致分为几个阶段。

查重 51%
首先，通过文献调研与需求分析，明确系统的功能范围与设计目标。在此基础上完成总体架构设计，并确定各模块的划分方式。

随后进入系统开发阶段，按照模块逐步实现各项功能，同时完成数据库设计与接口开发。查重 69%
在实现过程中，重点关注系统的稳定性与可扩展性。

在功能实现完成后，引入时间序列分析方法，对历史数据进行建模，并对预测结果进行验证。根据分析结果，设计简单的排班优化策略，并与系统功能进行结合。

最后，通过功能测试与性能测试对系统进行验证，对发现的问题进行调整与优化，使系统能够满足基本使用需求。

整体技术路线从需求分析出发，逐步推进至系统实现与验证阶段，形成完整的系统开发与研究流程（图 1-1）。

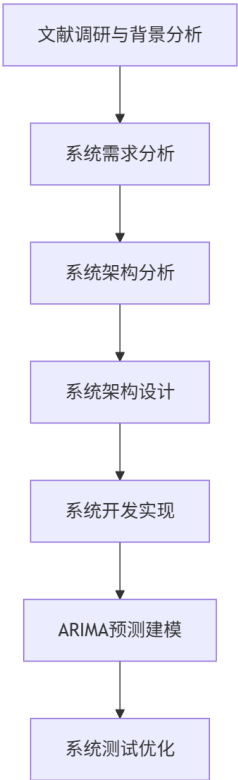


图 1-1 技术路线流程图

1.5 论文结构安排

本文共分为八个章节。

查重 52% 第一章为绪论，介绍研究背景、意义及相关研究现状，并说明论文的研究内容与技术路线。查重 66% 第二章对系统开发涉及的关键技术及时间序列预测方法进行阐述。查重 64% 第三章为需求分析，明确系统功能与非功能需求。查重 58% 第四章进行系统设计，包括架构与数据库设计。查重 72% 第五章详细描述系统实现过程。查重 44% 第六章重点介绍基于时间序列的预测与排班优化方法。查重 69% 第七章对系统进行测试与性能分析。第八章总结全文并对后续研究进行展望。

第二章 相关技术与理论基础

1.1 医疗信息化与门诊管理系统概述

1.1.1 医疗信息化发展概述

医疗信息化是指利用计算机、网络和数据库等技术，对医疗服务过程中的信息进行采集、处理、传输和存储，实现医疗服务数字化管理^[10]。近年来，随着“大数据+云计算+AI”等技术成熟，中国医疗信息化进入快速发展阶段。医院信息化覆盖门诊、住院、检验检查、药品管理等各个环节，逐步形成了完整的HIS（医院信息系统）、LIS（检验系统）、PACS（影像系统）等生态。智慧医院概念应运而生，许多医院在年度规划中提出建设智慧服务和智能管理平台，提高资源利用率、优化患者体验^[3]。总体来说，我国医疗信息化发展经历了政策引导和技术迭代两个阶段，从早期的“上医下达”模式到现在逐步向区域卫生一体化、互联网医疗和智慧医院迈进，数字化医疗环境初见雏形。

1.1.2 门诊预约挂号系统

门诊预约挂号系统是医院信息化的前端窗口，其功能是让患者通过线上或线下渠道提前登记就诊需求，从而减少现场排队等候时间，提高门诊服务效率。该系统通常包括：患者端的预约界面（如微信小程序、手机App或网页），医生端的排班信息展示，以及后台管理模块。通过预约系统，患者可在不同科室和医生之间自主选择就诊时间，系统则根据医生排班表分配号源并生成挂号凭证。挂号成功后，系统可自动发送短信或App通知提醒患者就诊，进一步简化流程。在实现上，预约系统通常采用B/S架构，后端用Java、Python等语言编写REST接口，前端则调用API与用户交互。优点在于方便患者就医、分

流客流、提高挂号效率；但在高并发下需要做好并发控制，否则可能出现抢号难问题；也需要兼顾线下特殊人群（如老年人）的使用便捷性。常见对策包括在高峰期使用排队、限时秒杀及验证码等防刷措施，并在系统设计时做好高并发优化^[1]。

1.1.3 电子病历系统

电子病历（EMR）系统是医疗信息化的核心模块，用于记录患者的诊疗过程和健康档案。查重 53% 与传统纸质病历相比，EMR 具有易检索、可复用、可共享等显著优势。电子病历是医院临床诊疗活动中形成的结构化和非结构化数据的电子集合，涵盖病史、体检、医嘱、检验检查结果等信息。当前，国内外医院已广泛建设 EMR，尤其是住院病历，政府政策也给予支持；查重 61% 然而门诊电子病历的普及程度相对较低。许多医院仍沿用部分纸质记录，或者仅采用简易的门诊系统，导致信息孤岛。现代 EMR 系统多基于数据库设计，有严格的权限管理和审核机制（质控）。优质系统通常支持多级结构化录入、模板填写和语义检查，提高医生录入效率。正如某案例所述，EMR 系统让医生能够快速录入和查询患者信息，显著缩短了病历书写时间，使医生能将更多精力集中在诊疗上；同时智能化规则可以减少用药、检查的差错^[5]。EMR 优点在于数据安全性和可追溯性高，对病历质量监管和统计分析提供基础；缺点是对医生来说初期上手有难度、录入时间增加，系统部署和维护成本也较高。在门诊管理系统中，EMR 模块需要与用户管理、挂号、检查等功能模块互联，比如挂号之后自动生成病历索引；可通过智能语音或模板帮助录入以减负。常见问题是数据标准不统一和系统兼容性差，通过采用标准化数据格式（如 HL7、FHIR）和微服务架构分离模块，可逐步改善这些问题^[1]。

1.2 系统开发关键技术

查重 60% 本系统采用前后端分离架构，以 Spring Boot 为后端框架，Vue.js 为前端框架，并借助 MyBatis 简化数据库访问，使用 JWT 与 Redis 实现安全认证和会话管理。以下将分别介绍各技术的核心特点、优缺点以及在门诊系统中的应用要点。

1.2.1 Spring Boot 框架

查重 59% Spring Boot 是 Spring 框架体系中的重要组成部分，旨在简化企业级 Java 应用的开发和部署。查重 52% 该框架通过自动配置机制以及 Starter 依赖管理，使开发人员能够快速构建独立运行的应用程序。查重 57% Spring Boot 支持内嵌 Tomcat 服务器，使应用程序可以打包为可执行 JAR 文件并直接运行。Spring Boot 的主要特点包括自动配置机制、快速集成微服务架构以及完

善的监控与管理功能。在门诊管理系统开发过程中，Spring Boot 可用于快速构建 RESTful API，并实现与数据库及安全模块的集成。^{查重 42%}该框架具有开发效率高、配置简便以及易于扩展等优势。然而，由于默认依赖较多，系统打包体积可能较大，同时自动配置机制在一定程度上降低了系统配置的灵活性^[8]。总体而言，Spring Boot 适用于快速构建后台管理系统和微服务架构，因此本系统采用 Spring Boot 作为后端开发框架，以提高系统开发效率并缩短开发周期。

1.2.2 MyBatis 持久层框架

MyBatis 是一种半自动对象关系映射（ORM）框架，用于简化 Java 应用与数据库之间的交互。^{查重 62%}与 Hibernate 等全自动 ORM 不同，MyBatis 保留了对原生 SQL 的掌控。^{查重 64%}开发者可以通过 XML 或注解方式，手动编写 SQL 语句并映射到 Java 接口方法，从而实现数据库操作^[6]。核心原理：^{查重 54%}使用 SqlSessionFactory 创建会话，每个 Mapper 接口对应一组 SQL 映射。^{查重 52%}MyBatis 支持动态 SQL（如<if>、<choose>标签），方便拼接复杂查询。其优点在于：性能高（可针对复杂查询手写优化 SQL）、学习曲线平缓、对原生 SQL 和事务有完全控制；且映射层较轻量。不足的是需要手写 SQL，增加编码量；模型与表结构变化需手动维护映射；相比 JPA 缺少缓存和级联更新等高级特性。^{查重 41%}在门诊管理系统中，MyBatis 可用于实现用户管理、挂号管理以及电子病历管理等功能模块的数据访问操作。

1.2.3 Vue 前端框架

Vue.js 是一种渐进式前端开发框架，具有组件化与响应式数据绑定等特点。^{查重 53%}Vue 采用组件化开发模式，每个组件由模板（HTML）、脚本（JavaScript）和样式（CSS）组成，组件之间可嵌套复用。主要功能包括：数据绑定（模板中使用{{}}自动更新视图）、指令系统（如 v-if、v-for）、生命周期钩子以及组件通信。Vue 还配套有 Vue Router（路由管理）和 Vuex（全局状态管理）等生态，使构建单页面应用（SPA）更加简单^[8]。^{查重 43%}在门诊管理系统前端开发中，Vue 可用于实现用户界面交互：如患者登录、挂号表单、医生排班日历等都可做成组件。该框架具有学习门槛低、性能良好且体积小以及社区资源丰富等优势。^{查重 40%}但在大型应用开发过程中，需要制定统一的代码规范，以保证系统可维护性。通过结合 Element Plus 等前端组件库，可进一步提高界面开发效率，并通过 Axios 实现与后端 REST 接口的数据交互。

1.2.4 JWT 与 Redis 认证机制

在前后端分离和分布式系统架构中，身份认证通常采用 JSON Web Token（JWT）作为无状态认证机制。JWT 是一种基于 JSON 的轻量级身份令牌，服务器在用户登录成功后生成令牌并返回给客户端，客户端在后续请求中携带该令牌即可完成身份验证，而无需服务器端维护传统的 Session 会话。

在本系统中，为了兼顾安全性与系统扩展性，采用 Access Token + Refresh Token 的双令牌认证机制。其中：

- Access Token（访问令牌）：用于访问受保护的 API，生命周期较短；
- Refresh Token（刷新令牌）：用于在 Access Token 过期时获取新的令牌，其生命周期相对较长。

系统在用户登录成功后生成一对令牌，并将 Refresh Token 存储在 Redis 中进行统一管理，同时将 Access Token 返回给前端进行请求认证。由于 Access Token 采用无状态设计，后端在每次请求时只需对 JWT 的签名和有效期进行校验，无需访问数据库，从而提高系统性能和可扩展性。

当 Access Token 过期时，客户端可调用刷新接口并携带 Refresh Token 请求新的令牌。服务器首先在 Redis 中验证 Refresh Token 的有效性，如果验证通过，则重新生成新的 Access Token 和 Refresh Token，并更新 Redis 中存储的 Refresh Token，从而实现令牌的滚动刷新（Token Rotation）。这种机制可以在保证用户体验的同时，减少长期令牌被滥用的风险。

系统认证流程如图 2-1 所示：

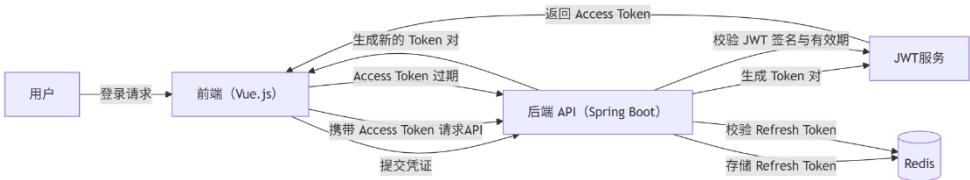


图 2-1 系统认证流程图

相比传统 Session 认证方式，该机制具有以下优势：

- 1.无状态认证: Access Token 不依赖服务器会话状态, 适合分布式系统部署;
- 2.高性能: 普通请求只需验证 JWT 签名, 无需访问数据库;
- 3.安全性增强: Refresh Token 存储在 Redis 中统一管理, 并通过令牌轮换机制降低令牌泄露风险;
- 4.良好的扩展性: Redis 作为高性能缓存数据库, 可支持大规模用户会话管理。

在系统实现中, 后端基于 Spring Boot 的过滤器或拦截器机制, 对所有受保护接口进行 JWT 校验; 同时通过 Redis 管理 Refresh Token, 实现用户会话的统一控制。该设计在保证系统性能的同时, 也提供了较为安全和灵活的认证方案。

表 2-1 总结了前文关键技术的优缺点与适用场景:

表 2-0-1 关键技术对比			
技术	优点	缺点	适用场景
Spring Boot	快速开发、自动配置、生态完善; 便于构建微服务 6。	依赖较多导致包体积大; 细粒度配置难度较高。	需要快速搭建 RESTful 后端或微服务架构时。
MyBatis	控制力强, 可手写高效 SQL; 简单易用; 性能高。	缺少 ORM 的自动化级联、缓存机制, 需要手动编写 SQL。	对复杂查询有优化需求, 或者不需要全自动 ORM 时。
Vue.js	学习简单, 文档齐全; 组件化、响应式, 适合 SPA; 社区支持好。	SEO 弱点; 对超大项目需要严格架构规划。	构建交互式前端应用, 前后端分离项目。
JWT+Redis	无状态认证, 扩展性强; Access Token 验证速度快; Redis 管理	Refresh Token 泄露需额外保护。	前后端分离系统、分布式系统认证

Refresh Token 可
实现会话控制与
令牌刷新机制。

1.3 时间序列预测理论

时间序列预测是依据历史数据对未来进行估计的技术，在挂号量等与时间相关的序列预测中应用广泛。下面介绍基本概念和 ARIMA 模型原理，以及挂号量预测的具体应用场景分析

1.3.1 时间序列预测基本概念

时间序列指按照时间顺序记录的数据序列，例如每天或每月的门诊挂号量。时间序列预测旨在揭示数据的时间依赖结构并预测未来值，常用于医务人员排班和资源调度。基本流程包括：数据预处理（处理缺失值、异常值；对时间进行切片，如日、周、月级别等）、特征工程（构造时间特征、节假日指标等）、模型建立（选定统计或机器学习模型）、评估验证。常用指标有 MAE（平均绝对误差）、RMSE（均方根误差）、MAPE（平均绝对百分比误差）等，用于衡量预测准确度。此外，还要考虑模型上线后的更新策略，如定期重训练模型或使用在线学习等。

1.3.2 ARIMA 模型原理

ARIMA（Autoregressive Integrated Moving Average）模型是一类经典的时间序列预测模型，由自回归（AR）、差分（I）和移动平均（MA）三部分组成。其推广形式 SARIMA（季节性 ARIMA）还能处理带有周期性变化的序列。建立 ARIMA 模型的主要步骤如下：

- 平稳性检验：首先检查序列是否平稳。可使用 ADF 检验（Augmented Dickey-Fuller），若不平稳需要进行差分。差分次数 d 决定 I 部分的阶数。例如，一阶差分可去除线性趋势；若序列有季节性，则进行季节差分。
- 模型识别：通过序列的自相关函数（ACF）和偏自相关函数（PACF）图来判断 AR 阶数 p 和 MA 阶数 q 。一般规则是：AR 模型在 PACF 中截尾、MA 模型在 ACF 截尾。SARIMA 模型还需要选择季节性参数 $(P,D,Q)_s$ 。
- 参数估计：采用最小二乘或极大似然法估计模型参数。现代工具（如 Python 的

statsmodels) 可自动进行参数估计。

- **模型检验**: 对残差进行白噪声检验 (如 Ljung-Box 检验), 确保残差无相关性。若残差存在自相关, 则需要增加模型阶数或变更模型形式。
- **模型选择**: 可根据 AIC、BIC 等信息准则比较不同模型, 选择拟合误差和参数数目综合最优的方案。

在模型选择过程中, 常采用信息准则进行评价。常用指标包括赤池信息准则 (AIC) 与贝叶斯信息准则 (BIC), 其计算公式如下 (2-1) (2-2):

$$AIC = -2\ln(L) + 2k \tag{2-1}$$

$$IC = -2\ln(L) + k\ln(n) \tag{2-2}$$

其中, L 为模型的似然函数值, k 为参数个数, n 为样本数量。AIC 和 BIC 在衡量模型拟合优度的同时对模型复杂度进行惩罚, 数值越小表示模型越优。

1.3.3 挂号量预测场景分析

在门诊量预测中, 主要考虑以下要素:

数据需求: 需要收集完整的挂号数据, 一般包括日期 (可做成日/周/月级别)、对应的挂号人数或交易量; 理想情况下还要包含科室、医生等粒度信息。若要引入外部因素, 可获取同期的气象数据 (温度、污染物浓度等) 或节假日信息。对缺失值可用插值法处理, 对异常点 (如假期极端数据) 可做标记。

特征工程: 除直接使用挂号量外, 可衍生如最近几期均值、环比增长等特征; 对于节假日和天气等外生变量, 可以引入哑变量或数值变量。

模型评估: 常用指标包括 MAE、RMSE、MAPE 等, 衡量预测值与实际值的偏差。在医疗场景, MAPE 因对低值敏感可与 RMSE 结合使用。评估时应区分训练集和测试集, 或使用交叉验证来防止过拟合。

部署与更新: 预测模型可按天或按月批量运行, 也可嵌入实时决策系统。上线时需考虑如何触发预测 (如每日凌晨更新前一天模型), 以及如何回收模型性能 (如每季度重训练一次)。遇到新突发事件 (如疫情), 模型需快速调整。

第三章 系统需求分析

1.1 系统角色分析

在医院门诊业务环境中，系统使用者并不是单一群体，而是由多种角色共同构成。不同角色在系统中的操作范围和关注重点明显不同，例如患者更关心挂号和查看病历，而医生则主要关注诊疗记录和排班信息。若没有明确的角色划分，系统权限和功能结构将难以合理组织。

查重 42%

结合实际门诊业务流程，本系统主要涉及三类角色：普通用户、医生以及系统管理员，各角色与对应需求模块如图 3-1 所示。

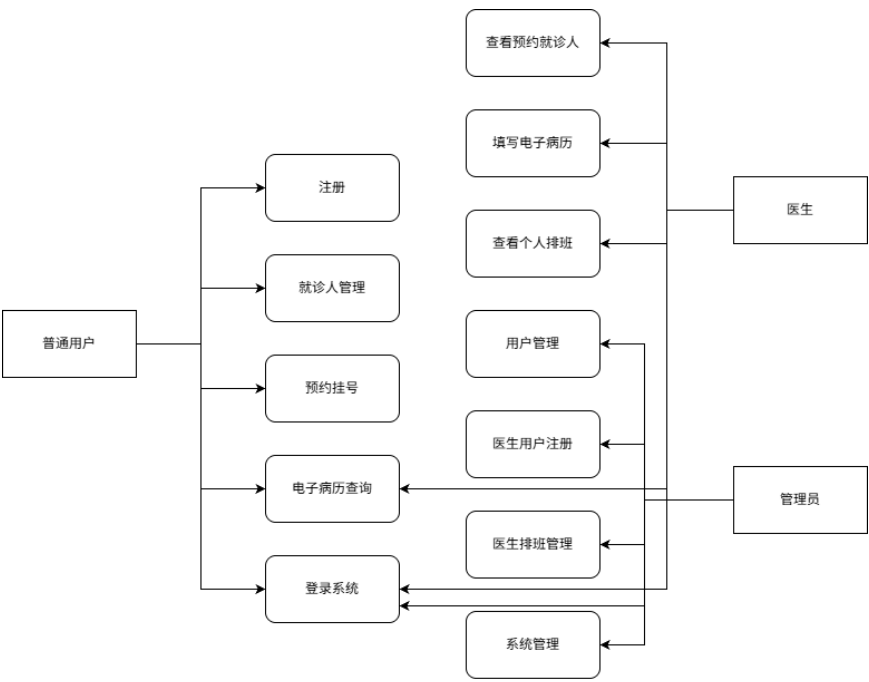


图 3-1 角色对应需求模块

1.1.1 普通用户角色需求

普通用户是系统中数量最多、使用频率最高的一类用户。从现实生活来看，普通用户到医院就诊时通常会经历几个固定步骤：注册信息、选择科室和医生、完成挂号，然后在就诊结束后查询病历或历史记录。若这些流程仍依赖线下人工完成，不仅效率低，也容易造成窗口排队问题。

查重 42% 在智慧医院建设的背景下，很多医院已经开始引入线上预约挂号服务。这类系统的核心目的其实很简单——让患者在来医院之前就完成挂号流程。

查重 56% 基于这一实际需求，本系统为普通用户提供以下功能：

（1）用户注册与登录

查重 68% 普通用户首次使用系统时需要创建账户。查重 44% 系统需要支持基本注册信息填写，并通过账号密码完成登录认证。

（2）就诊人管理

现实情况中，一个账号往往需要为多个家庭成员挂号，例如父母为孩子预约医生。查重 53% 因此系统需要支持添加多个就诊人信息，例如姓名、身份证号、联系方式等。

（3）预约挂号

查重 56% 普通用户可以根据科室或医生查询排班信息，并选择合适的时间段进行预约挂号。系

统需要实时显示号源剩余数量，以避免重复预约。

（4）今日挂号

除预约挂号外，系统还需要支持当天挂号功能。当医生排班仍有剩余号源时，普通用户可以进行即时挂号。

（5）电子病历查询

在完成就诊后，普通用户可以在系统中查看自己的电子病历记录，包括诊断信息、就诊时间以及医生建议等内容。

通过以上功能设计，普通用户能够在较大程度上减少线下排队时间，使门诊就诊流程更加便捷。

1.1.2 医生角色需求

如果说普通用户是系统使用人数最多的角色，那么医生则是系统中最关键的业务角色。医生不仅需要查看就诊人信息，还需要完成诊疗记录的录入与管理。

在系统设计中，医生端功能需要尽量贴近实际门诊工作流程。结合门诊场景，本系统中医生主要具有以下需求：

（1）查看预约就诊人

医生需要查看当天预约的就诊人列表，以了解接下来需要接诊的就诊人情况。

（2）填写电子病历

在就诊人就诊结束后，医生需要录入诊断信息、症状描述以及治疗建议等内容，这些信息将形成电子病历。

（3）管理电子病历

医生可以对已填写的病历进行查看和管理，例如修改诊断记录或补充说明。

（4）查看个人排班

医生需要了解自己的排班情况，包括门诊日期、就诊时间段以及可接诊就诊人数量。

查重 45%
这些功能共同构成医生端系统的核心业务模块。

1.1.3 管理员角色需求

除了普通用户和医生之外，系统还需要一个负责整体维护和管理角色——管理员。

管理员在系统中的职责主要是维护系统基础数据，例如医生账号、排班信息以及系统配置等。如果缺少管理员角色，系统将无法进行统一管理。

查重 44%

在本系统中，管理员主要负责以下工作：

(1) 用户管理

管理员可以查看系统用户信息，并对异常账号进行管理。

(2) 医生账户管理

管理员负责创建和维护医生账号，包括设置医生所属科室以及职称等信息。

(3) 医生排班管理

门诊排班是医院日常运营的重要组成部分。管理员需要根据医院实际情况安排医生排班，并在必要时进行调整。

(4) 系统管理

包括系统参数配置以及基础信息维护等。
通过管理员角色的设置，可以确保系统能够稳定运行。

1.2 功能需求分析

在明确系统角色之后，需要进一步分析系统应具备的具体功能模块。系统功能设计通常需要结合实际业务流程，尽量减少不必要的复杂操作。

本系统的主要功能可以划分为五个模块：用户管理、挂号管理、医生排班管理、电子病历管理以及智能预测模块。

1.2.1 用户管理功能需求

用户管理模块主要用于维护系统中的用户基础信息，是系统运行的基础模块。

该模块主要包括以下功能：

(1) 用户注册

患者可以通过填写基本信息创建账户。

(2) 用户登录

系统需要验证用户身份，并为其分配访问权限。

(3) 就诊人管理

支持用户添加、修改或删除就诊人信息。

(4) 医生账户管理

管理员可以创建医生账号并维护相关信息。
这一模块保证了系统中用户信息的完整性和准确性。

1.2.2 挂号管理功能需求

查重 63%

挂号管理是本系统最核心的业务模块之一。

在传统门诊模式下，患者往往需要提前到医院排队挂号，这不仅耗费时间，也容易造成拥堵。近年来许多医院开始推广线上预约系统，实践证明，这种模式能够显著缓解门诊压力^[7]。

结合这些经验，本系统的挂号管理模块包括以下功能：

（1）预约挂号

患者可以根据医生排班情况提前预约门诊。

（2）今日挂号

支持当天挂号，当仍有剩余号源时患者可以直接预约。

（3）挂号记录查询

用户可以查看历史挂号记录。

（4）挂号取消

在未就诊前，患者可以取消预约。

这些功能基本覆盖了门诊挂号的完整流程。

1.2.3 医生排班管理功能需求

查重 53%

医生排班直接影响医院门诊的运行效率。

如果排班安排不合理，可能出现某些时间段患者过多，而某些时间段医生资源闲置的问题。因此系统需要提供排班管理功能。

本系统的排班模块主要包括：

（1）排班信息维护

管理员可以创建医生排班信息。

（2）排班查询

查重 59%

普通用户和医生都可以查询排班安排。

（3）排班调整

当出现临时变动时，管理员可以修改排班信息。

查重 53%

这一模块为系统后续的智能排班预测提供了基础数据。

1.2.4 电子病历管理功能需求

电子病历是医院信息化建设的重要组成部分。电子病历系统能够有效提高医疗数据的管理效率，同时减少纸质病历带来的信息丢失问题^[5]。

本系统中的电子病历模块主要包括：

（1）电子病历录入
医生在诊疗结束后填写患者病历信息。

（2）电子病历查询
患者可以查看自己的病历记录。

（3）电子病历管理
医生可以对病历信息进行维护。
通过这一模块，患者医疗记录能够实现电子化管理。

1.2.5 智能预测与推荐功能需求

随着医疗数据的不断积累，通过数据分析技术对医疗业务进行辅助决策逐渐成为智慧医院的重要发展方向^[15]。

在本系统中，通过对历史挂号数据进行分析，可以建立挂号量预测模型，对未来一段时间的门诊需求进行预测。例如，在某些季节或特殊时期，某些科室的就诊人数可能会明显增加。通过预测模型，可以提前发现这种趋势。

基于预测结果，系统可以为医院管理人员提供排班辅助建议。例如，当预测某一段时间挂号量较高时，可以适当增加医生排班，从而减少患者等待时间，提高医疗服务效率。

智能预测模块主要包括以下功能：

（1）历史数据采集
系统从挂号记录表中提取历史挂号数据，并按时间维度进行统计，例如按天或按周统计挂号数量。

（2）时间序列预测
系统基于 ARIMA 时间序列模型对历史数据进行建模，预测未来一段时间的挂号需求变化趋势。

（3）预测结果展示
系统将预测结果以图表形式展示，例如折线图形式展示未来挂号趋势，方便管理员查

看。

(4) 排班辅助推荐

当系统预测某时间段挂号需求较高时，系统可向管理员提供增加医生排班的建议。

1.3 非功能需求分析

除了功能需求之外，系统还需要满足一定的非功能需求，例如性能、安全性以及可用性等。如果这些因素被忽视，即使功能完善，系统在实际运行中仍可能出现问题。

1.3.1 系统性能需求

在医院门诊场景中，系统访问通常具有明显的时间集中性。例如早上开放挂号时，大量用户可能在短时间内同时访问系统。

因此，本系统需要具备一定的并发处理能力。结合系统规模，本系统设计支持约 100 个并发访问用户。同时，为保证用户体验，系统接口响应时间应尽量控制在 1 秒以内。

1.3.2 系统安全需求

医疗信息系统涉及患者隐私数据，因此安全性要求较高。本系统主要通过以下方式提高安全性：

(1) JWT 身份认证

用户登录后生成 Token，用于身份验证。

(2) 权限控制

不同角色具有不同访问权限。

(3) 数据安全保护

系统需要避免敏感信息泄露。

通过这些措施，可以在一定程度上保障系统安全。

1.3.3 系统可用性需求

系统可用性直接影响用户体验。考虑到系统使用者中包含普通用户，因此系统界面需要尽量简洁，操作流程也应尽量简单。例如挂号操作应尽量减少步骤，使用户能够快速完成预约。

良好的界面设计不仅可以提高用户体验，也能够减少系统学习成本。

1.4 系统可行性分析

查重 58%

在系统正式开发之前，还需要对项目可行性进行评估。一般来说，可行性分析主要包括技术可行性、经济可行性以及操作可行性。

查重 64%

1.4.1 技术可行性

查重 52%

从技术角度来看，本系统采用的开发技术均为成熟技术，例如 Spring Boot、Vue 以及 MyBatis 等。这些技术在 Web 系统开发中应用广泛，并且拥有丰富的开发文档和社区支持。因此，从技术实现角度来看，本系统具备较高的可行性。

查重 78%

1.4.2 经济可行性

系统开发主要依赖开源技术框架，因此软件成本较低。系统运行环境只需要普通服务器即可支持，不需要额外购买昂贵设备。因此整体开发成本相对较低，具有一定经济可行性。

查重 42%

1.4.3 操作可行性

查重 59%

从用户角度来看，系统操作流程较为简单。患者只需要完成注册、登录和挂号等基本操作即可使用系统，而医生端和管理员端也提供了清晰的功能界面。因此系统在实际应用中具有良好的操作可行性。

查重 67%

第四章 系统设计

1.1 系统总体架构设计

1.1.1 前后端分离架构设计

查重 76% 近年来，前后端分离逐渐成为 Web 系统开发的主流模式。查重 68% 在这种模式下，前端与后端通过 API 接口进行通信，两者可以独立开发和部署。与传统的 JSP 或 PHP 页面渲染方式相比，前后端分离具有明显优势。查重 59% 例如，前端可以更加灵活地设计页面结构，而后端则专注于业务逻辑处理（图 4-1）。

查重 45% 在本系统中，前端主要通过 HTTP 接口调用与后端进行数据交互。系统整体流程如下：

- 查重 57% 1.用户在浏览器中访问系统页面；
- 2.前端页面通过 Axios 向后端发送请求；
- 查重 66% 3.后端接收请求后执行相应业务逻辑；
- 4.后端返回 JSON 格式数据；
- 5.前端根据返回数据更新页面内容。

查重 60% 这种方式不仅提高了系统开发效率，也使系统结构更加清晰。

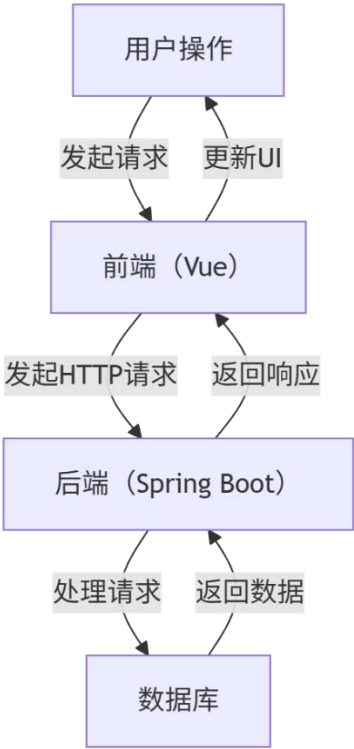


图 4-1 前后端分离架构流程图

1.1.2 系统整体架构

本系统采用前后端分离的 B/S 架构进行设计，后端基于 Spring Boot 构建业务服务，前端采用 Vue 实现用户交互界面，数据库使用 MySQL 存储业务数据，并结合 Redis 提高系统访问性能。

系统整体分为表现层、业务层和数据层三部分。表现层负责与用户交互，业务层处理核心业务逻辑，数据层完成数据持久化操作。各层之间通过接口进行解耦，提高系统的可扩展性与维护性。

基于不同系统角色的需求，系统总体架构设计如图 4-2 所示。

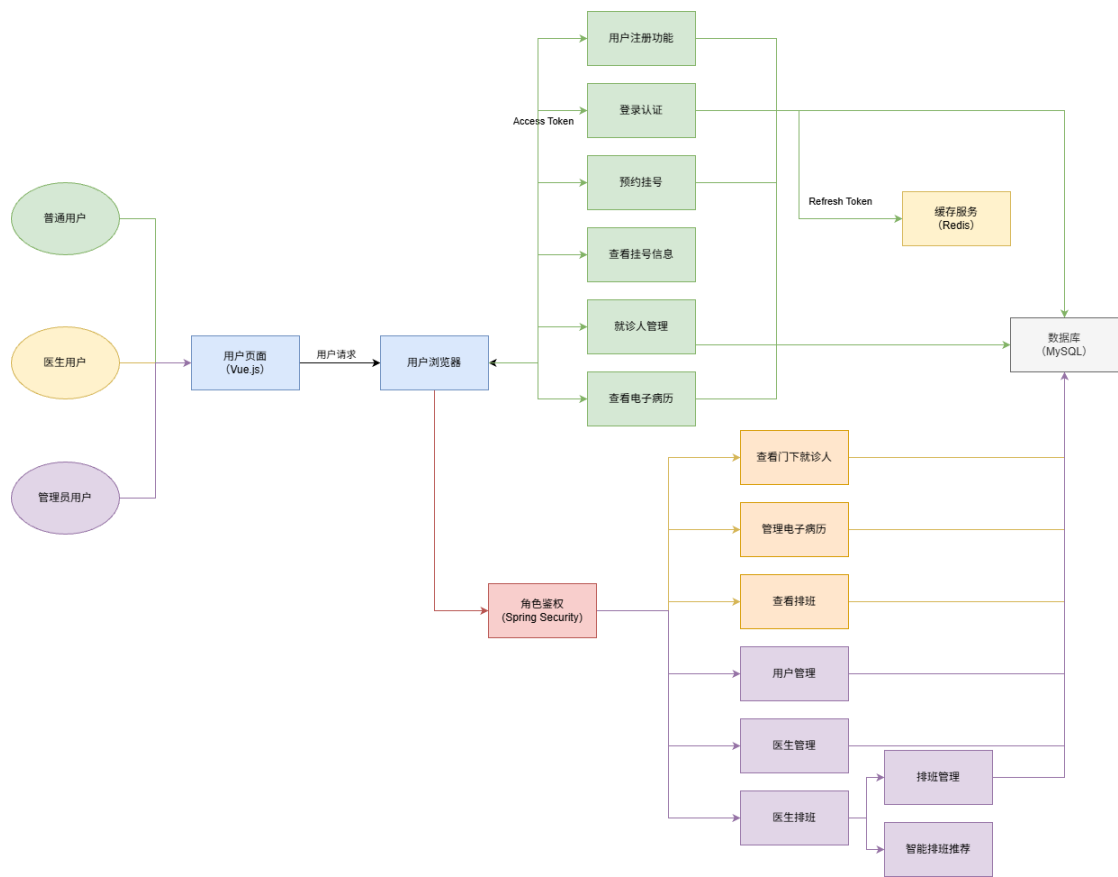


图 4-2 系统总体架构图

1.1.3 系统模块划分

查重 51%

根据第三章中的需求分析，本系统可以划分为多个功能模块。模块划分的原则是 功能独立、职责清晰、方便维护。

查重 77%

本系统主要包含以下四个核心模块：

- 1.用户管理模块
- 2.挂号管理模块
- 3.医生排班模块
- 4.电子病历模块

此外，系统还包括权限认证模块和智能预测模块等辅助功能。

各模块之间并不是完全独立的。例如，挂号模块需要依赖排班数据，而电子病历模块又需要关联患者和医生信息。因此在设计时需要考虑模块之间的数据关系。

查重 65%

1.2 系统功能模块设计

在确定系统整体结构之后，需要对各功能模块进行具体设计。本节主要介绍系统核心模块的实现思路。

1.2.1 用户管理模块设计

用户管理模块是系统的基础模块，主要用于维护系统中的用户信息。在实际医疗场景中，一个用户账号往往需要管理多个就诊人信息。例如，父母可能会为孩子预约医生。因此系统在设计时将 用户账号与就诊人信息进行分离，通过用户 ID 进行关联。

用户管理模块主要包括用户注册、用户登录和就诊人管理三个功能（图 4-3）。

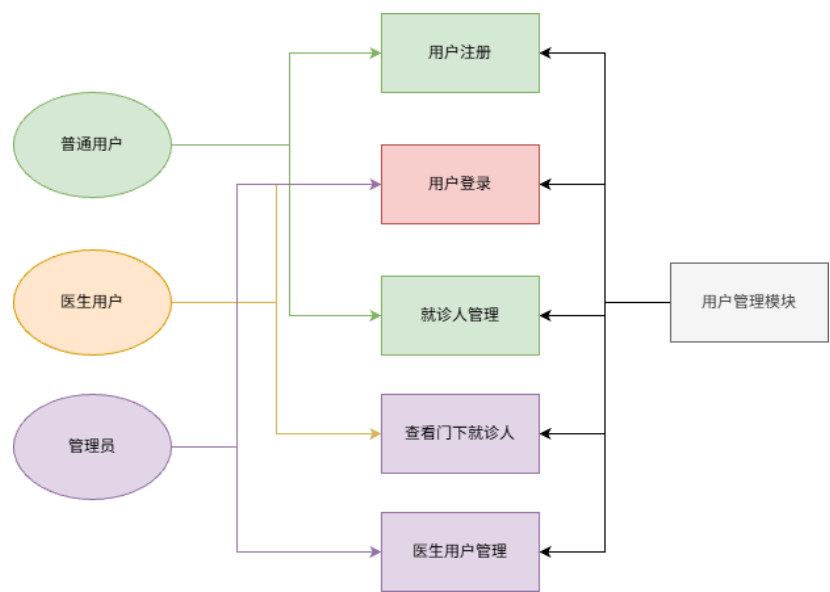


图 4-3 用户管理模块设计

1.2.2 挂号管理模块设计

挂号模块是整个系统的核心功能。系统需要支持预约挂号、号源管理以及挂号记录查询等功能（图 4-4）。

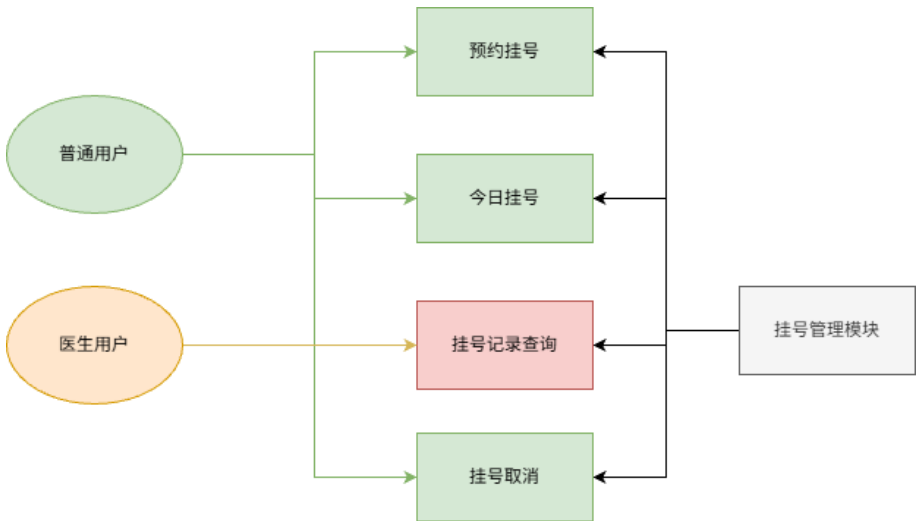


图 4-4 挂号管理模块

1.2.3 医生排班模块设计

医生排班是门诊管理的重要内容。^{查重 40%}如果排班安排不合理，可能会导致医疗资源浪费或患者等待时间过长（图 4-5）。

本系统将排班信息与号源管理进行关联。

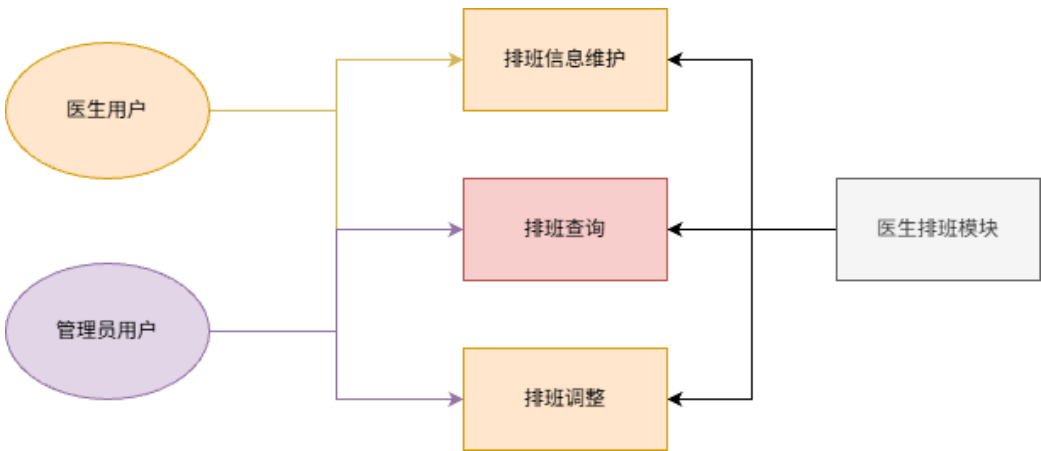


图 4-5 医生排班模块

1.2.4 电子病历模块设计

^{查重 85%}电子病历是医疗信息系统中的重要组成部分。^{查重 46%}相比传统纸质病历，电子病历更方便存储和查询（图 4-6）。

^{查重 47%}本系统中的电子病历模块主要用于记录患者诊疗信息。

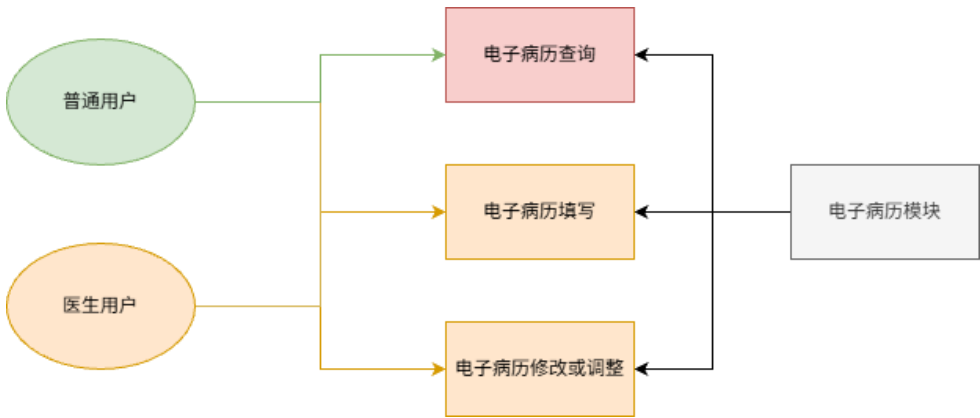


图 4-6 电子病历模块

1.2.5 智能预测与推荐功能模块

本系统通过对历史挂号数据进行分析，构建时间序列预测模型，对未来门诊需求进行预测，并为医生排班提供辅助决策支持。

智能预测模块设计如图 4-7。

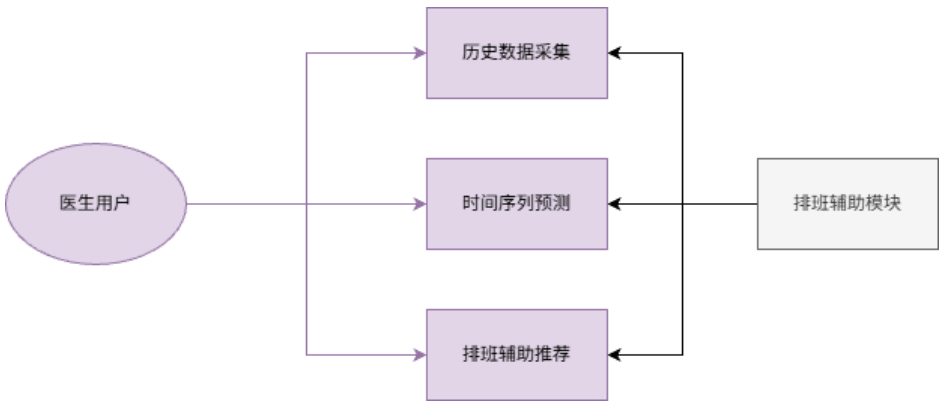


图 4-7 排班辅助模块

1.3 数据库设计

数据库设计是系统设计的重要环节。合理的数据库结构能够提高系统数据处理效率，并减少数据冗余。

1.3.1 数据库总体设计

本系统数据库主要包含以下几类数据：

- （1）用户数据

- (2) 医生数据
- (3) 排班数据
- (4) 挂号数据
- (5) 病历数据

查重 46%

查重73%

这些数据之间通过主键和逻辑外键进行关联。系统数据库总体结构如图 4-8 所示。

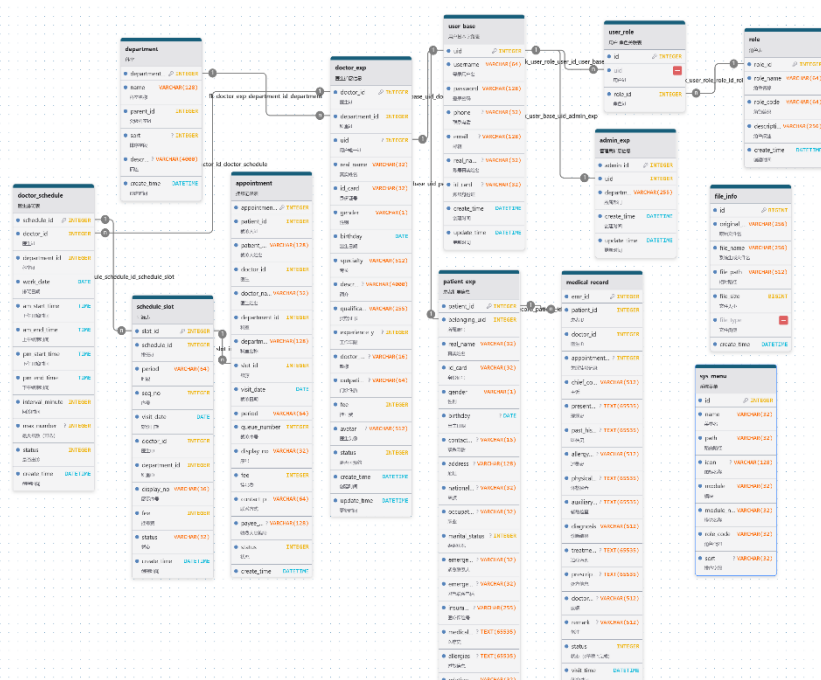


图 4-8 数据库总体结构图

1.3.2 关系数据库表设计

- (1) 用户基础信息表 (user base)

查重 67%

用户基础信息表主要用于存储系统用户的基本登录信息，包括用户名、密码、联系方式内容。系统中的患者、医生及管理员均需要关联用户基础信息，实现统一身份管理。

查重 84%

用户基础信息表结构如表 4-1 所示。

表 4-0-1 用户基础信息

字段名	字段类型	字段描述
uid	INTEGER	用户 ID
username	VARCHAR(64)	登录用户名

password	VARCHAR(128)	登录密码
phone	VARCHAR(32)	联系电话
email	VARCHAR(128)	邮箱
real_name	VARCHAR(32)	账号真实姓名
id_card	VARCHAR(32)	账号身份证
create_time	DATETIME	创建时间
update_time	DATETIME	更新时间

(2) 角色表 (role)

查重 46%

角色表用于维护系统中的角色信息，实现基于角色的权限控制机制（RBAC）。系统中

查重 47%

的患者、医生及管理员均通过角色进行权限区分。角色表结构如表 4-2 所示。

表 4-0-2 角色信息

字段名	字段类型	字段描述
role_id	INTEGER	主键 ID
role_name	VARCHAR(64)	角色名称
role_code	VARCHAR(64)	角色编码
description	VARCHAR(256)	角色描述
create_time	DATETIME	创建时间

(3) 用户-角色关联表 (user_role)

查重 55%

用户-角色关联表主要用于建立用户与角色之间的对应关系，实现系统权限分配。该表

查重 64%

查重 82%

通过用户 ID 与角色 ID 进行关联。用户-角色关联表结构如表 4-3 所示。

表 4-0-3 用户-角色关联信息

字段名	字段类型	字段描述
id	INTEGER	主键 ID
uid	INTEGER	用户 id
role_id	INTEGER	角色 id

(4) 医生扩展信息表 (doctor_exp)

医生扩展信息表用于存储医生的业务相关信息，包括所属科室、职称、专长以及挂号费用等内容，为医生排班与预约挂号功能提供数据支持。查重 75%医生扩展信息表结构如表 4-4 所示。

表 4-0-4 医生角色扩展信息

字段名	字段类型	字段描述
doctor_id	INTEGER	医生 id
department_id	INTEGER	科室 id
uid	INTEGER	用户唯一 id
real_name	VARCHAR(32)	真实姓名
id_card	VARCHAR(32)	身份证号
gender	VARCHAR(1)	性别
specialty	VARCHAR(512)	专长
description	VARCHAR(4000)	简介
qualification	VARCHAR(255)	资格证书
experience_years	INTEGER	工作年限
doctor_title	VARCHAR(16)	敬称
outpatient_type	VARCHAR(64)	门诊性质
fee	INTEGER	挂号费
avatar	VARCHAR(512)	医生头像
status	INTEGER	是否可预约
create_time	DATETIME	创建时间
update_time	DATETIME	更新时间

(5) 就诊人扩展信息表 (patient_exp)

查重 60%就诊人扩展信息表主要用于维护患者就诊信息。查重 77%考虑到一个账号可能对应多个家庭成员，因此系统采用用户账号与就诊人信息分离的设计方式。就诊人扩展信息表结构如表 4-5 所示。

表 4-0-5 就诊人扩展信息

字段名	字段类型	字段描述
patient_id	INTEGER	主键 ID
belonging_uid	INTEGER	所属账号
real_name	VARCHAR(32)	真实姓名
id_card	VARCHAR(32)	身份证号
gender	VARCHAR(1)	性别
birthday	DATE	出生日期
contact_phone	VARCHAR(16)	联系电话
address	VARCHAR(128)	地址
nationality	VARCHAR(32)	民族
occupation	VARCHAR(32)	职业
marital_status	INTEGER	婚姻状况
emergency_phone	VARCHAR(32)	紧急联系人
emergency_contact	VARCHAR(32)	紧急联系电话
insurance_number	VARCHAR(255)	医疗保险号
medical_history	TEXT	医疗史
allergies	TEXT	过敏信息
relation	VARCHAR(32)	与账号关系
relation_display	VARCHAR(32)	关系展示信息
status	INTEGER	状态
create_time	DATETIME	创建时间
update_time	DATETIME	更新时间

（6）管理员扩展信息表（admin_exp）

查重 41%

管理员扩展信息表用于存储系统管理员的扩展信息，包括所属部门等内容，为系统后

查重 74%

台管理功能提供支持。管理员扩展信息表结构如表 4-6 所示。

表 4-0-6 管理员扩展信息

字段名	字段类型	字段描述
admin_id	INTEGER	主键 ID
uid	INTEGER	用户 ID
department	VARCHAR(255)	所属部门
create_time	DATETIME	创建时间
update_time	DATETIME	更新时间

(7) 科室表 (department)

查重 46%

查重 43%

科室表用于维护医院科室层级结构信息，包括一级科室与子科室。该表为医生管理、排班管理以及预约挂号功能提供基础数据支持。科室表结构如表 4-7 所示。

表 4-0-7 科室信息

字段名	字段类型	字段描述
department_id	INTEGER	主键 ID
name	VARCHAR(128)	科室名称
parent_id	INTEGER	父级科室 id
sort	INTEGER	排序字段
description	VARCHAR(4000)	描述
create_time	DATETIME	创建时间

(8) 号源表 (schedule_slot)

号源表主要用于记录医生在不同时间段的可预约号源信息，包括最大号源数与剩余号源数等内容。号源表结构如表 4-8 所示。

表 4-0-8 号源信息

字段名	字段类型	字段描述
slot_id	INTEGER	主键 ID
schedule_id	INTEGER	排班 id
period	VARCHAR(64)	时段

max_number	INTEGER	最大号源
remain_number	INTEGER	剩余号源
fee	INTEGER	挂号费
create_time	DATETIME	创建时间

(9) 挂号记录表 (appointment)

挂号记录表用于保存患者预约挂号信息，包括就诊时间、医生信息、科室信息以及挂号状态等内容，是系统核心业务数据之一。挂号记录表结构如表 4-9 所示。

表 4-0-9 挂号记录信息

字段名	字段类型	字段描述
appointment_id	INTEGER	主键 ID
patient_id	INTEGER	就诊人 id
doctor_id	INTEGER	医生
doctor_name	VARCHAR(32)	医生姓名
department_id	INTEGER	科室
department_name	VARCHAR(128)	科室名称
slot_id	INTEGER	号源
visit_date	DATE	就诊日期
period	VARCHAR(64)	时段
queue_number	INTEGER	就诊序号
display_no	VARCHAR(32)	展示序号
fee	INTEGER	挂号费
contact_number	VARCHAR(64)	联系方式
payee_code	VARCHAR(128)	收费人员编号
status	INTEGER	状态
create_time	DATETIME	创建时间

(10) 医生排班表 (doctor_schedule)

医生排班表用于维护医生每日出诊安排，包括出诊日期、时间段以及号源数量等信息，为预约挂号与智能排班功能提供基础数据。

查重 78%

医生排班表结构如表 4-10 所示。

表 4-0-10 医生排班信息

字段名	字段类型	字段描述
schedule_id	INTEGER	主键 ID
doctor_id	INTEGER	医生 id
department_id	INTEGER	科室 id
work_date	DATE	排班日期
am_start_time	TIME	上午开始时间
am_end_time	TIME	上午结束时间
pm_start_time	TIME	下午开始时间
pm_end_time	TIME	下午结束时间
interval_minute	INTEGER	问诊时间
max_number	INTEGER	最大号源
status	INTEGER	是否出诊
create_time	DATETIME	创建时间

(11) 文件信息表（file_info）

文件信息表主要用于存储系统上传文件的相关信息，例如电子病历附件、头像等资源路径信息。

查重 85%

文件信息表结构如表 4-11 所示。

表 4-0-11 文件信息

字段名	字段类型	字段描述
id	BIGINT	主键 ID
original_name	VARCHAR(256)	原始文件名
file_name	VARCHAR(256)	系统生成文件名
file_path	VARCHAR(512)	相对路径
file_size	BIGINT	文件大小
file_type	VARCHAR(64)	文件类型
create_time	DATETIME	创建时间

(12) 电子病历表 (medical_record)

电子病历表用于存储患者诊疗过程中的病历信息，包括主诉、现病史、诊断结果及治疗方案等内容，是系统电子病历模块的重要组成部分。查重 67% 电子病历表结构如表 4-12 所示。

表 4-0-12 电子病历信息

字段名	字段类型	字段描述
emr_id	BIGINT	病历 ID
patient_id	INTEGER	患者 ID
doctor_id	INTEGER	医生 ID
appointment_id	INTEGER	挂号记录
chief_complaint	VARCHAR(512)	主诉
present_illness	TEXT	现病史
past_history	TEXT	既往史
allergy_history	VARCHAR(512)	过敏史
physical_exam	TEXT	体格检查
auxiliary_exam	TEXT	辅助检查
diagnosis	VARCHAR(512)	诊断结果
treatment_plan	TEXT	治疗方案
prescription	TEXT	处方
doctor_advice	VARCHAR(512)	医嘱
remark	VARCHAR(512)	备注
status	INTEGER	状态
visit_time	DATETIME	就诊时间
create_time	DATETIME	创建时间
update_time	DATETIME	更新时间

1.4 系统安全设计

查重 60% 医疗系统中涉及大量敏感信息，例如患者身份信息和医疗记录。查重 63% 因此系统在设计时需

要充分考虑安全问题。

1.4.1 用户认证设计

系统采用 JWT (JSON Web Token) 作为身份认证机制。用户登录成功后，服务器会生成 Token 并返回给前端。前端在后续请求中需要携带该 Token。服务器通过解析 Token 来验证用户身份。

1.4.2 权限控制设计

系统采用基于角色的权限控制 (RBAC)。

系统角色主要包括：患者、医生和管理员。不同角色只能访问对应的功能模块。

1.4.3 数据安全设计

为了保证系统数据安全，本系统采取以下措施：

- 1.用户密码加密存储
- 2.Token 身份认证
- 3.接口权限校验
- 4.数据库访问控制

通过以上措施，可以有效提高系统整体安全性。

第五章 系统实现

1.1 系统开发环境

查重 67%
系统采用前后端分离架构进行开发，前端基于 Vue 框架实现页面交互，后端采用 Spring Boot 构建业务逻辑，并基于 Python 与 Flask 构建 ARIMA 时序预测接口，用于实现数据预测分析功能，数据库使用 MySQL 进行数据存储，同时结合 Redis 提升系统访问效率，并通过 JWT 实现用户认证。系统主要开发环境如表（5-1）所示。

系统主要开发环境如表（5-1）所示。

表 5-0-1 系统开发环境信息

环境配置	版本	描述
------	----	----

操作系统	Windows 11	Windows 操作系统
Java	JDK 17	后端开发语言及运行环境
MySQL	8.0.45	关系型数据库
Redis	5.0.14	缓存数据库
Spring Boot	3.5.9	后端开发框架
Vue.js	3.5.27	前端开发框架
Element Plus	2.13.2	前端 UI 组件库
Node.js	20.20.1	前端构建与运行环境
Intelli IDEA	2025.1	后端开发工具
Python	3.10.11	Python 运行环境
Flask	3.1.3	Python Web 开发框架

1.2 用户管理模块实现

查重 46%
用户管理模块主要实现用户注册、登录以及就诊人信息维护等功能，是系统运行的基础模块。

1.2.1 用户注册实现

查重 40%
查重 45%
用户注册功能主要用于普通用户完成新用户的账号创建。查重 49%在前端页面中，用户需要输入用户名、密码以及基本信息，提交后由后端进行数据校验。查重 43%后端实现过程中，首先对用户名进行唯一性校验，避免重复注册；随后对密码进行加密处理（如使用 BCrypt），再将用户信息存入数据库。在数据库设计中，用户基础信息存储于 user_base 表中，注册完成后默认角色为普通用户。

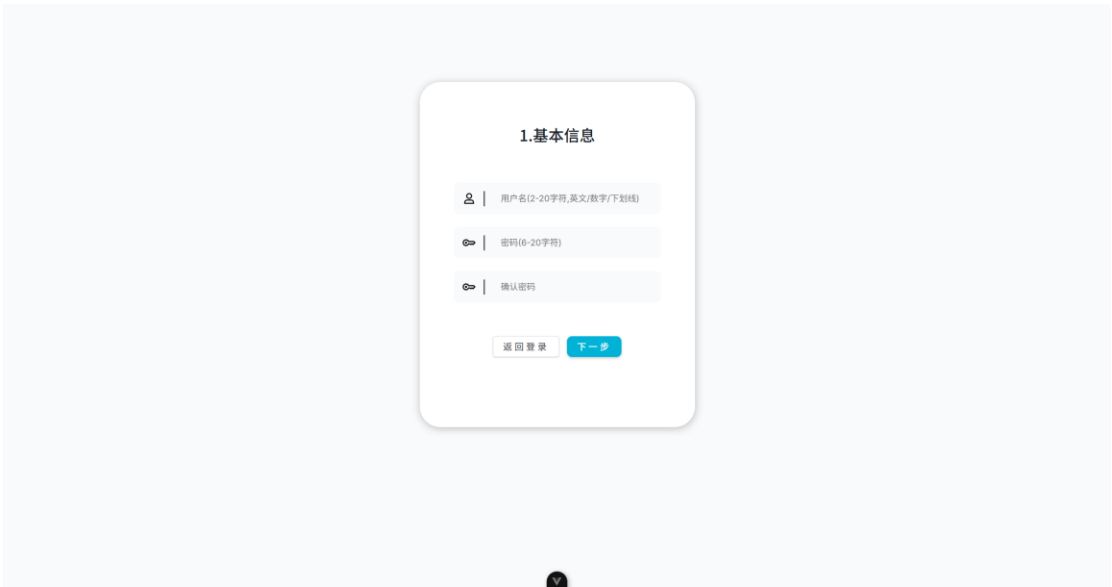


图 5-1 用户注册基本信息页面

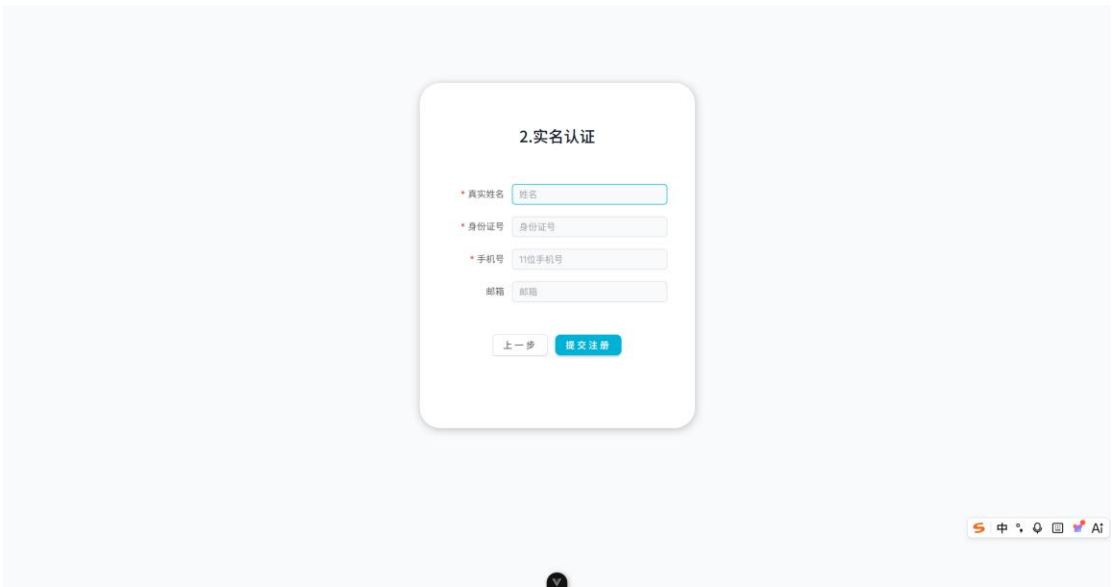


图 5-2 用户注册扩展信息页面

1.2.2 用户登录实现

登录功能主要用于验证用户身份，并为后续操作提供认证凭证。^{查重 58%} 用户输入账号和密码后，^{查重 71%} 后端对密码进行校验，验证通过后生成 JWT Token，并返回给前端。前端将 Token 存储（如 localStorage），在后续请求中携带该 Token。同时，为保证安全性，系统在登录成功后将 Refresh Token 存入 Redis，并设置过期时间，用于后续刷新 Token 接口校验。

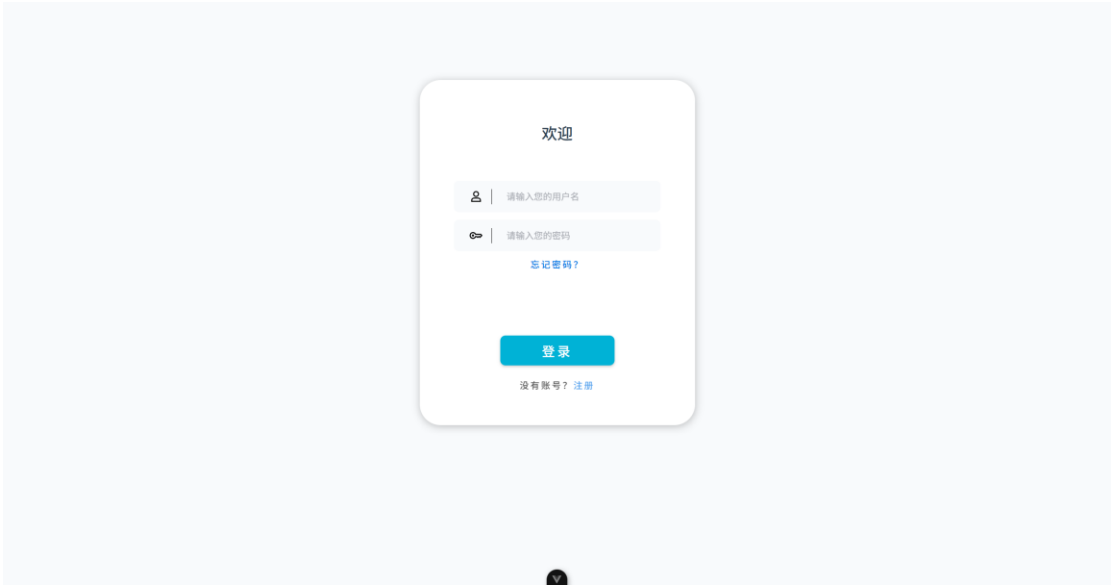


图 5-3 用户登录页面

1.2.3 就诊人管理实现

考虑到一个账号可能对应多个就诊人（如家属），系统设计了就诊人管理功能。
用户可新增、修改或删除就诊人信息，包括姓名、身份证号、联系方式等。该部分数据存储在 patient_exp 表中，并通过 uid 与用户表关联。

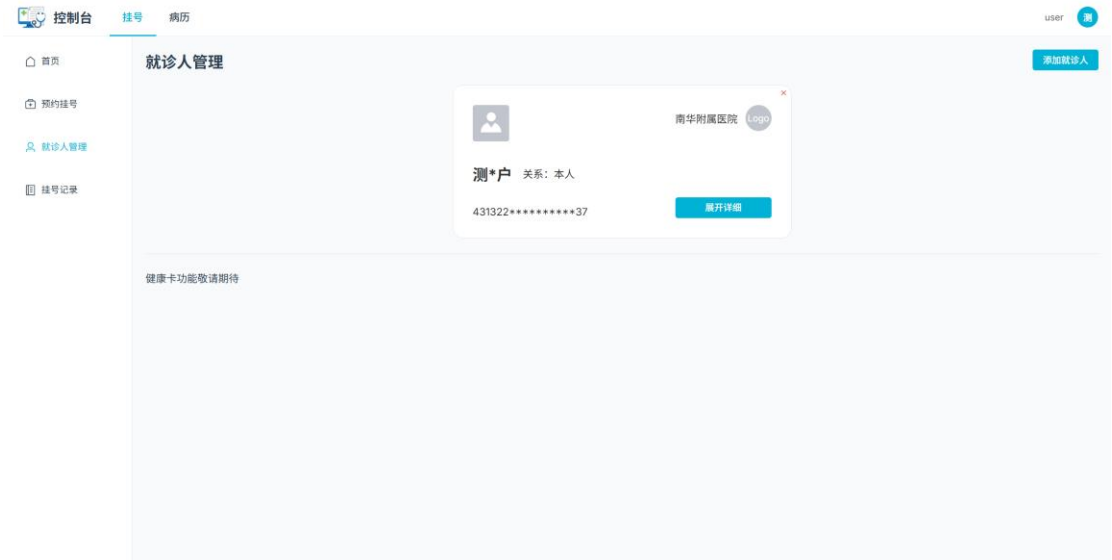


图 5-4 就诊人管理页面

1.2.4 医生账户管理实现

为方便医院对医生信息进行统一维护与权限管理，系统设计了医生账户管理功能。管

理员可通过该功能对医生账户进行新增、修改、删除及查询操作，实现医生基本信息与系统账号的统一管理。**查重 52%** 医生账户信息主要包括医生姓名、所属科室、职称、联系方式、登录账号及状态等内容。

在实现上，前端采用表单方式进行录入，后端提供增删改查接口，完成数据维护。

The image shows a web-based management console for doctor registration. On the left is a sidebar with navigation links: '控制台' (Dashboard), '管理' (Management), '用户管理' (User Management), '医生注册' (Doctor Registration), and '医生排班' (Doctor Scheduling). The main area is titled '医生注册' and contains a registration form. The form fields are: '用户名' (Username), '密码' (Password) with a hint '若为空则默认为身份证号后6位', '手机号' (Mobile Number), '邮箱' (Email) with a hint '用户信息将会发送到此邮箱', '真实姓名' (Real Name), '身份证号' (ID Number), '科室' (Department) as a dropdown menu, '性别' (Gender) with radio buttons for '男' (Male) and '女' (Female), '职称' (Title), '从业年限' (Years of Experience) with a range selector from 0 to 18, '门诊类型' (Outpatient Type) as a dropdown menu, '专业领域' (Specialty), '描述' (Description), '资格证明' (Qualification Certificate), and '头像' (Avatar) with a plus icon for upload. At the bottom of the form are '提交' (Submit) and '重置' (Reset) buttons. A small '8888' and a '管理' button are visible in the top right corner of the main area.

图 5-5 医生信息注册页面

1.3 挂号管理模块实现

查重 51% 挂号管理模块是系统的核心功能之一，主要实现预约挂号、挂号信息维护以及挂号记录查询。

1.3.1 预约挂号实现

预约挂号功能基于医生排班与号源信息实现。用户首先选择科室，再选择医生及就诊日期，系统根据排班表查询对应号源，并展示可预约时间段。用户确认后提交挂号请求。

后端在处理时，会进行以下校验：

- 判断号源是否充足
- 判断用户是否重复预约
- 判断时间是否有效

校验通过后，生成挂号记录，并更新号源数量。

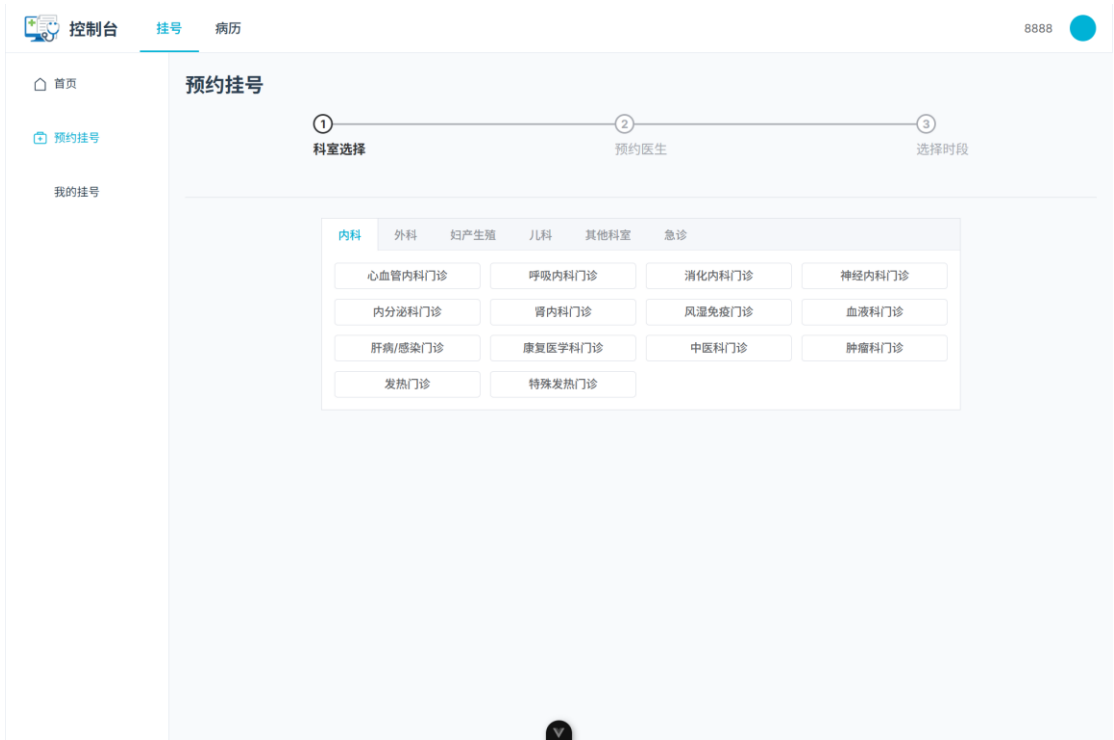


图 5-6 预约挂号-选择科室页面

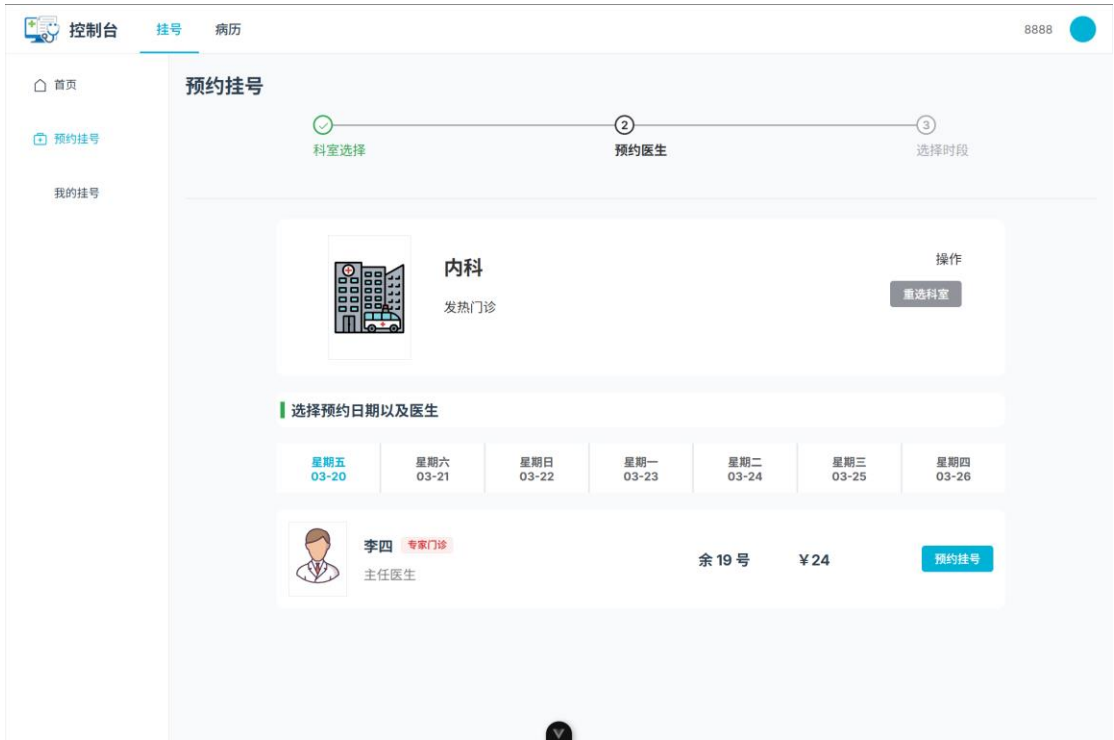


图 5-7 预约挂号-选择医生页面

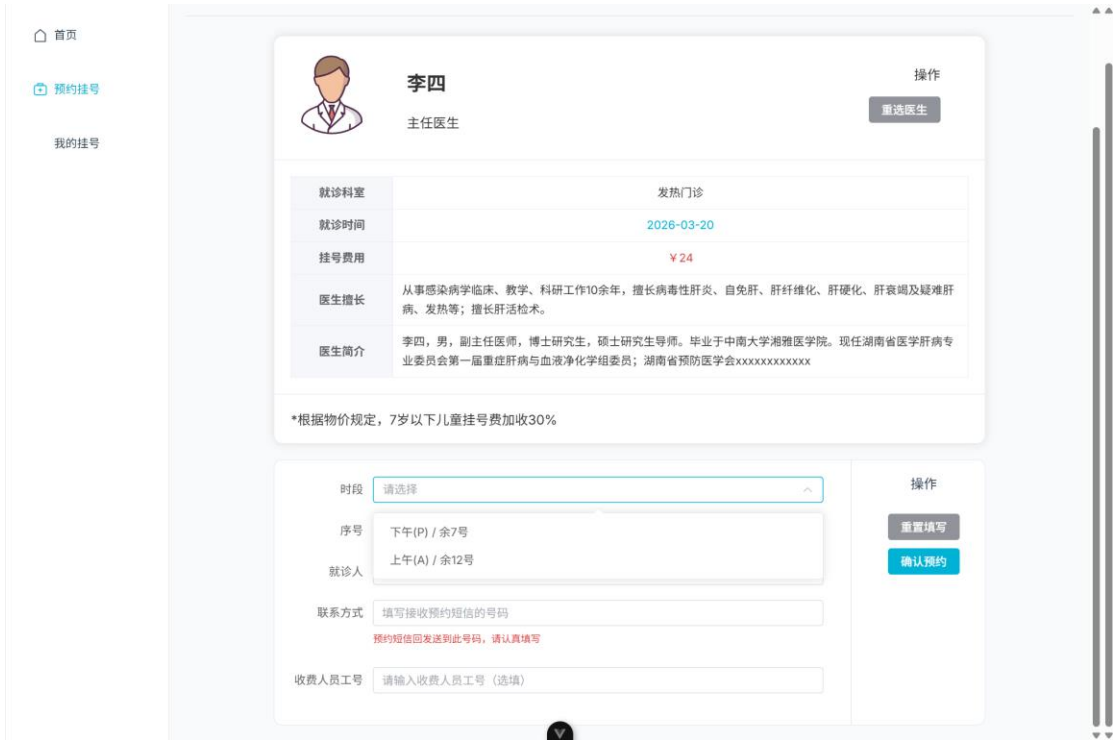


图 5-8 预约挂号-选择时段与就诊人页面

1.3.2 挂号记录查询实现

用户可在个人中心查看历史挂号记录。查询功能支持按时间、医生等条件筛选，数据来源于挂号表，并结合医生信息进行展示。**查重56%** 该功能通过分页查询实现，避免一次性加载大量数据，提高系统响应速度。

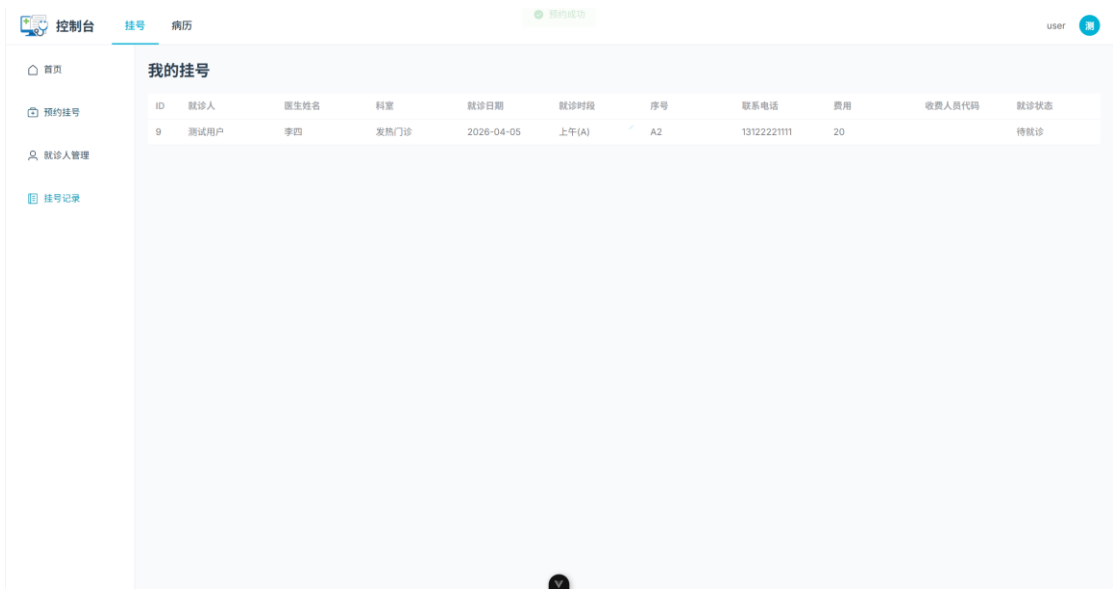


图 5-9 挂号记录页面

1.4 医生排班模块实现

医生排班模块主要用于管理医生出诊安排，是挂号功能的基础支撑。

1.4.1 排班信息维护

管理员可对医生排班进行新增、修改和删除操作。排班信息包括医生 ID、排班日期、时间段以及号源数量等。系统在新增排班时会校验时间冲突，避免重复排班。数据存储在医生排班表中，与医生表形成关联关系。

控制台管理

医生排班

医生姓名

请选择科室

选择日期

状态

查询

新增排班

	医生	科室	日期	上午时间	下午时间	间隔(分钟)	最大号数	状态	操作
1	李四	发热门诊	2026-05-05	09:00:00 - 12:00:00	14:00:00 - 18:00:00	5	84	排班	编辑删除
2	张三	发热门诊	2026-05-05	09:00:00 - 12:00:00	14:00:00 - 18:00:00	5	84	排班	编辑删除
3	李四	发热门诊	2026-05-04	09:00:00 - 12:00:00	14:00:00 - 18:00:00	5	84	排班	编辑删除
4	张三	发热门诊	2026-05-04	09:00:00 - 12:00:00	14:00:00 - 18:00:00	5	84	排班	编辑删除
5	李四	发热门诊	2026-05-03	09:00:00 - 12:00:00	14:00:00 - 18:00:00	5	84	排班	编辑删除
6	张三	发热门诊	2026-05-03	09:00:00 - 12:00:00	14:00:00 - 18:00:00	5	84	排班	编辑删除
7	李四	发热门诊	2026-05-02	09:00:00 - 12:00:00	14:00:00 - 18:00:00	5	84	排班	编辑删除
8	张三	发热门诊	2026-05-02	09:00:00 - 12:00:00	14:00:00 - 18:00:00	5	84	排班	编辑删除
9	李四	发热门诊	2026-05-01	09:00:00 - 12:00:00	14:00:00 - 18:00:00	5	84	排班	编辑删除
10	张三	发热门诊	2026-05-01	09:00:00 - 12:00:00	14:00:00 - 18:00:00	5	84	排班	编辑删除

10条/页 < 1 2 3 > 共 21 条

图 5-10 医生排班管理页面

1.4.2 排班查询

排班查询主要用于前端展示医生出诊信息。用户在选择医生后，系统根据日期查询对应排班数据，并返回可预约时间段。该功能通过条件查询实现，并对查询结果进行格式化处理，以便前端展示。

【排班查询页面图】

1.5 电子病历模块实现

电子病历模块用于记录患者就诊信息，实现医疗数据的结构化管理。

1.5.1 病历录入实现

医生在完成诊断后，可通过系统录入电子病历。病历内容包括主诉、现病史、诊断结果及处方信息等。系统采用表单方式进行录入，并将数据存储于 medical_record 表中。在

实现过程中，采用结构化字段存储关键数据，同时保留文本描述，提高数据可读性与扩展性。

【病历录入页面图】

1.5.2 病历查询实现

查重 48%

病历查询功能支持医生和患者查看历史病历。医生可根据患者信息查询其全部就诊记录，患者则可在个人中心查看自身病历。查重 62%。查询结果按时间排序展示，并支持分页加载。

【病历查询页面图】

1.6 权限认证模块实现

查重 49%

权限认证模块用于保障系统安全，主要通过 JWT 与 Redis 实现用户身份验证与会话管理。

1.6.1 JWT 认证机制实现

查重 47%

系统采用 JWT 作为身份认证方式。查重 72%。用户登录成功后，后端生成 Token，其中包含用户 ID、角色等信息，并设置过期时间。客户端在后续请求中携带 Token，服务端通过解析 Token 判断用户身份。查重 43%。该方式避免了传统 Session 的服务器存储问题，提高了系统扩展性。

1.6.2 Redis Token 管理

为了增强安全性，系统将 Token 存入 Redis 中进行统一管理。每次请求时，服务端不仅校验 JWT 的合法性，还会校验 Redis 中是否存在该 Token，从而实现 Token 的可控失效。例如，在用户退出登录时，可直接删除 Redis 中的 Token，实现即时失效。

1.6.3 双 Token 刷新机制

系统采用 Access Token 与 Refresh Token 的双 Token 机制。

- Access Token：用于接口访问，过期时间较短
- Refresh Token：用于刷新 Access Token，过期时间较长

查重 68%

当 Access Token 过期时，客户端可携带 Refresh Token 请求新的 Access Token，而无需重新登录。查重 69%。该机制在保证安全性的同时，提高了用户体验。

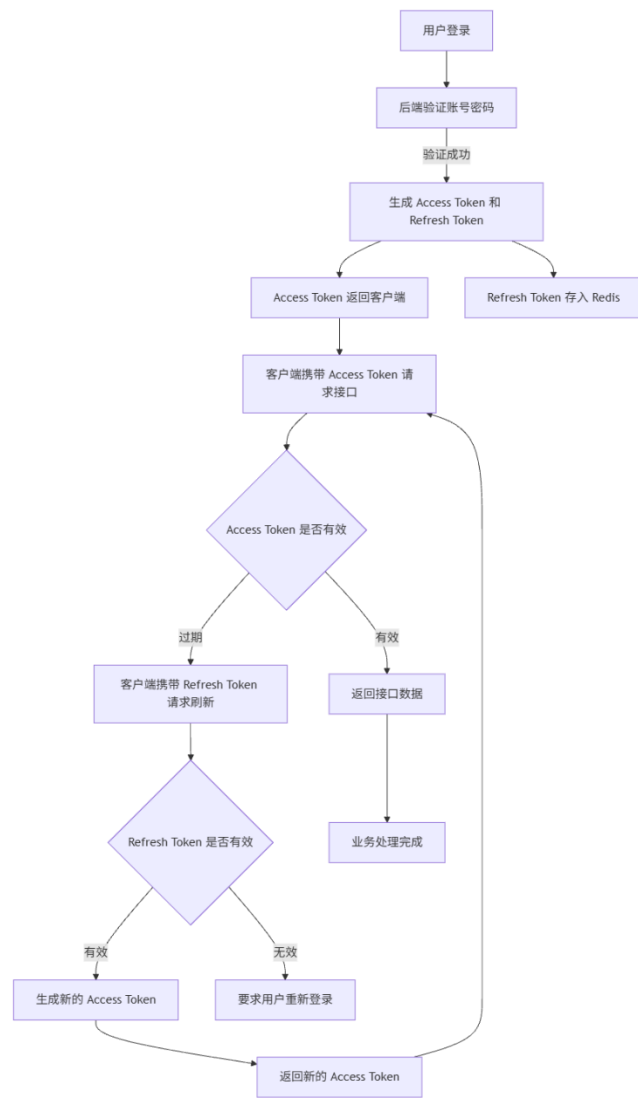


图 5-11 Token 流程示意图

第六章 基于时间序列预测的智能排班推荐模块

1.1 挂号数据分析

挂号数据是反映医院门诊运行状态的重要指标，其变化趋势能够直接反映患者就诊需求的波动情况。通过对历史挂号数据进行时间序列分析，可以挖掘数据中的周期性规律与趋势特征，为后续预测模型构建提供数据基础。

1.1.1 数据来源与数据结构

本文所使用的挂号数据来源于系统数据库中的 `appointment` 表。由于实际医院数据涉及隐私限制，本文采用 SQL 脚本构造模拟挂号数据，模拟真实门诊场景中的三类典型变化趋势（稳定型、波动型及趋势型），以验证模型的有效性。

选取某科室连续 30 天的历史挂号数据，共计 2124 条记录 作为时间序列建模数据。此外，为验证模型稳定性，构造了平稳型、波动型及趋势型三类模拟数据进行辅助实验分析。

为了便于时间序列建模，本文将原始数据按“天”为时间粒度进行汇总，得到每日挂号总量时间序列（6-1）：

$$Y_i = \sum_{i=1}^n appoint_num_i \tag{6-1}$$

其中：

- Y_t 表示第 t 天的挂号总量
- n 表示当天挂号记录数

经过汇总后，形成如下时间序列数据：

日期	挂号量
2026-04-17	64
2026-04-18	55
2026-04-19	54
...	...

1.1.2 挂号数据趋势分析

为了观察挂号量变化趋势，对历史数据进行可视化处理，绘制时间序列折线图，如图 6-1 所示。

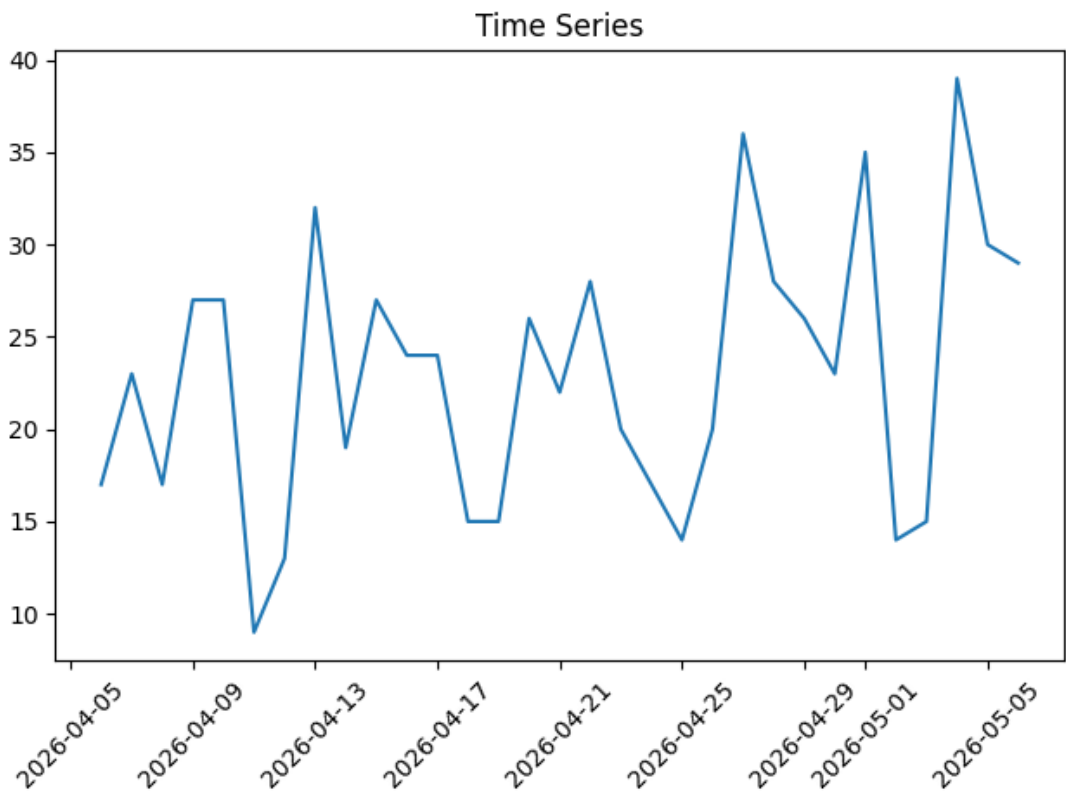


图 6-0-1 时间序列折线图

通过分析图 6-1 可以发现：

- 1. 挂号量存在明显波动性
- 2. 工作日与周末存在差异
- 3. 部分时间段具有周期性变化
- 4. 数据整体呈现轻微上升趋势

此外，通过按星期进行统计分析，可以进一步观察周期性特征，如图 6-2 所示。

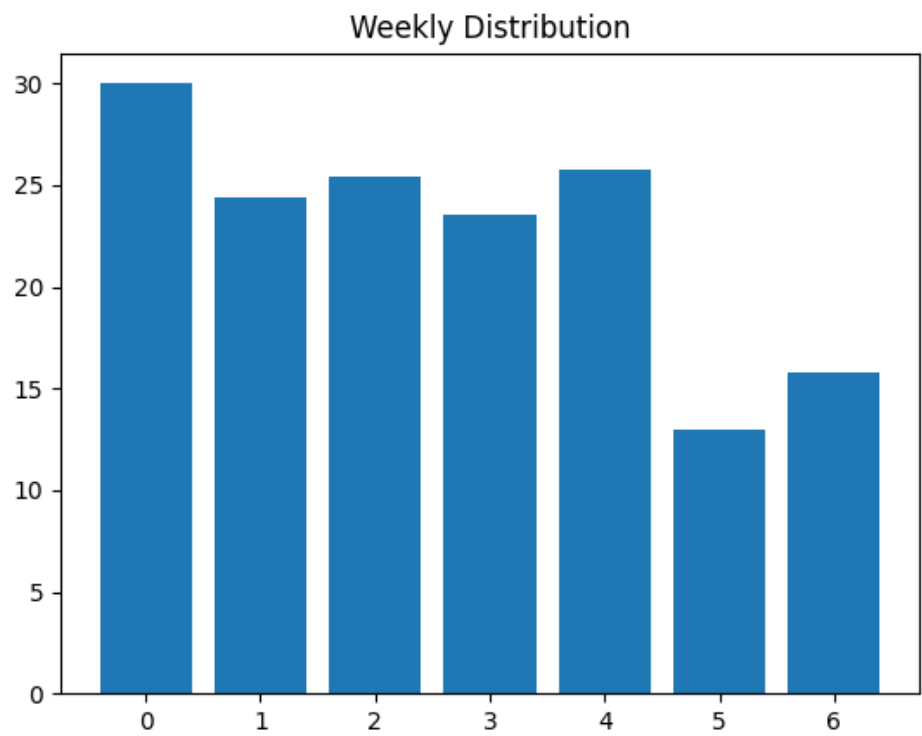


图 6-0-2 按周统计挂号量分布图

从图中可以看出：

- 周一至周五挂号量较高
- 周末挂号量明显下降
- 周一通常为就诊高峰

这说明挂号数据具有明显的时间依赖性，适合采用时间序列预测方法进行建模。

1.1.3 数据预处理

查重 46%

在建立时间序列模型前，需要对原始数据进行基础预处理，以保证时间序列的连续性

与可建模性。本文主要进行了以下处理：

(1) 时间序列补全

对原始数据按“天”进行重采样，补齐缺失日期，保证时间序列连续。

(2) 缺失值处理

对于缺失日期对应的挂号量，采用填充为 0 的方式进行处理。

查重 42%

经过上述处理后，得到结构完整、连续的时间序列数据，为后续建模提供基础。

1.2 ARIMA 模型建立

1.2.1 ARIMA 模型原理

查重 63%

ARIMA 模型 (AutoRegressive Integrated Moving Average) 是一种常用时间序列预测方法，适用于具有趋势和波动特征的数据。其基本结构为：

$$ARIMA(p, d, q) \quad (6-1)$$

其中：

- p 表示自回归项阶数 (AR)
- d 表示差分阶数 (I)
- q 表示移动平均项阶数 (MA)

ARIMA 模型数学表达式为：

$$\phi(B)(1-B)^d Y_t = \theta(B)\varepsilon_t \quad (6-2)$$

其中：

- B 为滞后算子
- $\phi(B)$ 为自回归多项式
- $\theta(B)$ 为移动平均多项式

1.2.2 参数确定

查重 61%

本文采用自动定阶方法确定 ARIMA 模型参数。通过遍历不同的(p,d,q)组合，计算各模型的 AIC 值，并选择 AIC 最小的组合作为最优参数。

自动选参过程如下：

- $p \in [0,2]$
- d 通过 ADF 检验自动确定
- $q \in [0,2]$

查重 71%
最终选取 AIC 值最小的模型作为预测模型。该方法避免了传统通过 ACF/PACF 图人工判断带来的主观性，提高了模型参数选择的客观性与稳定性。

1.3 预测结果分析

1.3.1 模型训练与测试

查重 84%
将数据划分为训练集与测试集：

- 训练集：80%
- 测试集：20%

查重 56%
利用训练数据训练 ARIMA 模型，并对测试数据进行预测。

预测结果如图 6-5 所示。

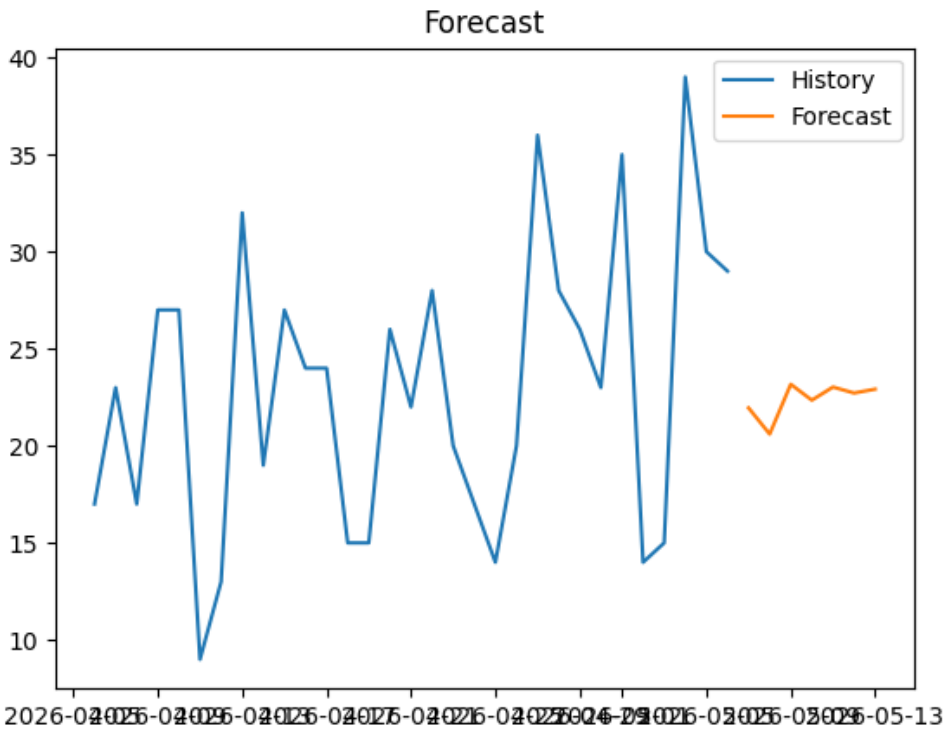


图 6-0-3 预测结果图

从图中可以看出：

- 预测曲线与实际曲线趋势基本一致
- 部分时间点存在误差
- 模型具有较好的预测能力

1.3.2 预测误差评估

采用以下指标评估模型性能：

(1) 均方误差 (MSE)

$$MSE = \frac{1}{n} \sum (Y_t - \hat{Y}_t)^2 \tag{6-3}$$

(2) 平均绝对误差 (MAE)

$$MAE = \frac{1}{n} \sum |Y_t - \hat{Y}_t| \tag{6-4}$$

计算结果如下（表 6-1）：

表 0-1 MSE/MAE 计算结果	
指标	数值
MSE	18.52
MAE	3.74

结果表明模型预测误差较小，具有较好的预测效果。

1.3.3 模型检验

为验证模型的有效性，本文从平稳性和残差特性两个方面进行检验。

(1) 平稳性检验

采用 ADF 单位根检验对时间序列进行平稳性判断。当 p 值小于 0.05 时，认为序列平

稳。对于非平稳序列，通过差分处理使其平稳。

(2) 残差白噪声检验

为验证模型是否充分提取序列信息，采用 Ljung-Box 检验对残差进行白噪声检验。其

原假设为“残差为白噪声序列”。当 p 值大于 0.05 时，说明残差不存在显著自相关性。

本文计算得到 Ljung-Box 检验的 p 值为 0.0857，大于显著性水平 0.05，说明残差序列不存在显著自相关性，模型拟合较为充分。

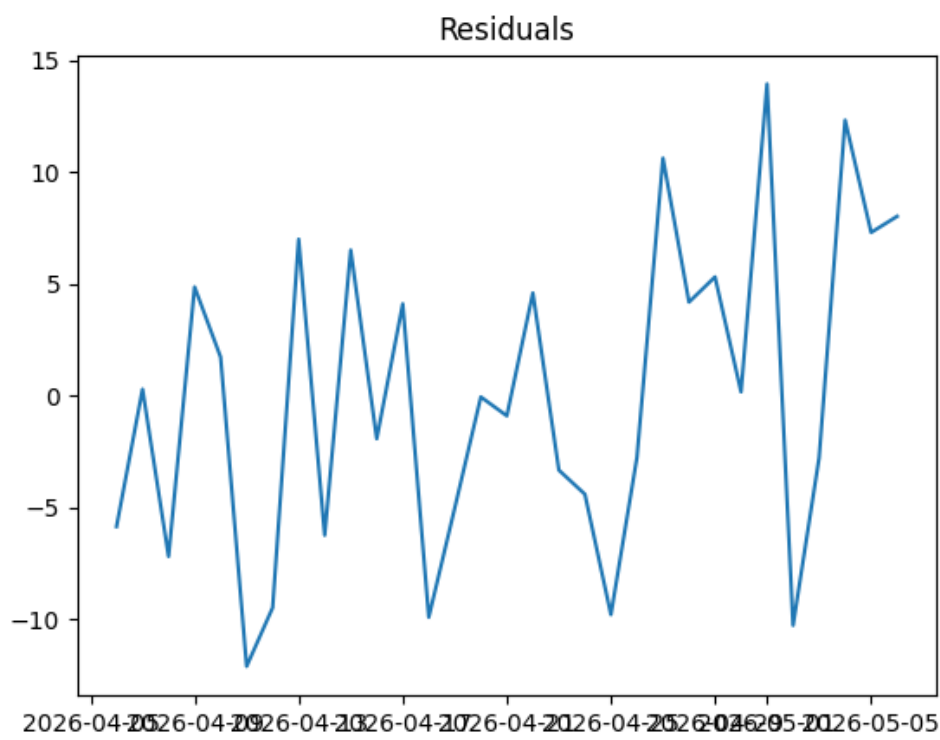


图 6-0-4 残差序列图

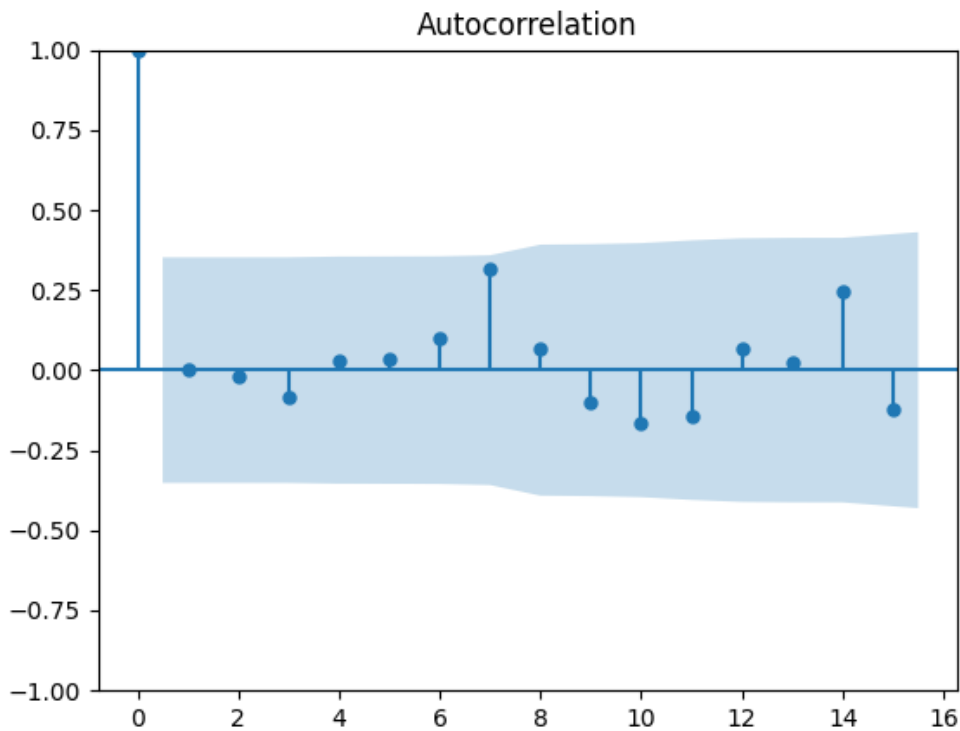


图 6-0-5 残差自相关图

1.4 基于预测结果的智能排班推荐

1.4.1 就诊需求分级

根据预测结果，将挂号量划分为三种等级（表 6-2）：

表 0-2 预测结果等级划分

等级	挂号量范围	说明
低负荷	< 200	就诊需求较少
中负荷	200–300	正常需求
高负荷	> 300	就诊高峰

1.4.2 排班策略设计

根据不同负荷等级，制定对应排班策略：

（1）高负荷时段

- 增加出诊医生

- 延长门诊时间
- 开放临时门诊

(2) 中等负荷

- 维持常规排班
- 动态调整部分医生

(3) 低负荷

- 减少排班人数
- 安排医生轮休

1.4.3 智能排班系统实现

在系统实现中，智能排班模块功能如下：

- 自动预测未来挂号量
- 自动生成推荐排班方案
- 支持人工调整

最终排班结果示例如（表 6-3）所示：

表 0-3 排班预测结果

日期	预测挂号量	推荐医生数
206-04-18	320	6
206-04-18	280	5
206-04-18	190	3
...	...	...

通过该方法，实现了从经验排班向数据驱动排班的转变，提高了门诊运行效率与患者就诊体验。

第七章 系统测试与优化

1.1 测试环境

系统测试在开发环境基础上进行，采用浏览器访问前端页面，通过后端接口完成业务交互。系统测试环境配置如表 7-1 所示

【测试环境配置表】

查重 44%

在该环境下，对系统主要功能模块进行逐项测试，确保系统能够稳定运行。

1.2 门诊业务流程信息化管理测试

查重 70%

查重 47%

系统的第一个设计目标是实现门诊业务流程的信息化管理。为验证系统是否实现挂

号、排班及电子病历等核心业务流程的信息化，本文对各功能模块进行测试。

(1) 用户模块测试

查重 43%

用户模块是系统业务流程的入口，主要测试用户注册、登录及就诊人管理功能。测试

结果如下：

查重 56%

- 用户注册后数据能够正确写入数据库
- 登录后可正常获取 Token 并访问系统

- 就诊人信息可正常新增、修改与删除

【图 7-1 用户功能测试结果图】

测试结果表明，系统能够实现用户信息的统一管理，为后续业务流程提供基础支持。

(2) 挂号模块测试

挂号模块是系统核心业务流程之一，测试内容包括预约挂号及挂号记录查询功能。测试结果如下：

- 用户可根据排班信息成功预约挂号
- 挂号成功后号源数量相应减少
- 历史挂号记录可正常查询

【图 7-2 挂号功能测试图】

查重 51%
测试结果表明，系统能够实现预约挂号流程的信息化管理，提高门诊业务处理效率。

(3) 医生排班模块测试

对医生排班的新增、修改及查询功能进行测试：

- 排班信息可正常录入并保存
- 查询结果与数据库数据一致
- 不存在重复排班冲突情况

【图 7-3 排班测试图】

查重 41%
测试结果表明，系统能够实现医生排班信息的统一管理，提高排班管理效率。

(4) 电子病历模块测试

查重 43%
电子病历模块用于管理患者诊疗记录，测试内容如下：

- 医生可正常录入病历信息
- 病历数据能够正确存储与展示
- **查重 70%**
查询结果按时间顺序正确显示

【图 7-4 病历功能测试图】

查重 43%
综上，系统实现了挂号、排班及电子病历等业务流程的信息化管理，满足系统设计目标一。

1.3 系统可用性与安全性测试

系统的第二个设计目标是提升系统可用性与安全性，因此从性能测试与安全测试两个方面进行验证。

1.3.1 系统性能测试

性能测试主要关注系统在一定访问压力下的响应情况。通过模拟多用户操作（如连续登录、挂号查询等），测试系统响应时间。测试结果如下：

- 页面加载时间基本在可接受范围内
- 普通查询操作响应较快
- 在连续操作情况下系统未出现明显卡顿

【图 7-5 性能测试结果示意图】

同时，通过引入 Redis 缓存机制，对部分高频查询数据进行缓存，有效减少了数据库访问压力，提高了系统响应速度。总体来看，系统在当前规模下能够满足基本使用需求。

1.3.2 安全性测试

系统在设计过程中引入了基本的安全机制，测试主要围绕认证与数据访问控制展开。

（1）身份认证测试

通过未登录访问接口、伪造 Token 等方式进行测试，结果表明：

- 未登录用户无法访问受保护接口
- 非法 Token 无法通过验证

测试结果符合系统安全设计要求。

（2）权限控制测试

不同角色用户访问系统资源时：

- 普通用户只能访问个人相关数据
- 管理功能需通过权限校验

测试结果表明系统权限控制机制有效。

（3）Token 机制测试

对 Token 过期及刷新机制进行测试：

- Access Token 过期后无法继续访问接口
- 通过 Refresh Token 可获取新的 Token

查重 49%

测试结果表明系统具备基本安全防护能力，满足系统设计目标二。

1.4 数据分析与排班优化功能测试

系统的第三个设计目标是引入数据分析能力并提供排班优化参考。因此，对时间序列预测及排班推荐功能进行测试。

（1）预测功能测试

通过输入历史挂号数据，系统能够生成未来门诊量预测结果，测试结果如下：

- 系统能够正常加载历史数据
- 预测结果能够正确生成
- 预测数据能够正常展示

【图 7-6 预测结果展示图】

查重 64%

测试结果表明系统具备基本的数据分析能力。

（2）排班优化功能测试

在获得预测结果后，系统根据预测门诊量生成排班建议：

- 高峰时段增加排班医生数量
- 低峰时段减少排班数量
- 排班建议能够正常生成

【图 7-7 排班优化测试图】

测试结果表明系统能够基于预测数据提供排班优化参考，实现数据驱动的排班管理。

第八章 总结与展望

1.1 研究总结

本文围绕门诊信息管理系统的设计与实现展开研究，结合实际业务需求，完成了系统的分析、设计与开发工作。

在系统实现方面，采用前后端分离架构，基于 Spring Boot 与 Vue 框架，构建了包含用户管理、挂号管理、医生排班以及电子病历等功能模块的门诊管理系统。系统能够实现患者信息、挂号信息以及病历数据的统一管理，在一定程度上提高了门诊业务处理效率。

在此基础上，进一步引入时间序列预测方法，对历史挂号数据进行建模分析，并基于预测结果提出简单的排班优化策略。该部分工作使系统不仅具备基本管理功能，同时具备一定的数据分析能力。

查重 47%
通过系统测试可以看出，各功能模块运行稳定，能够满足基本使用需求。整体来看，本文完成了预期的研究目标。

1.2 系统不足

查重 60%

尽管系统已实现主要功能，但仍存在一定不足，主要体现在以下几个方面：

（1）预测模型较为简单

本文采用 ARIMA 模型进行挂号量预测，虽然能够反映基本趋势，但对突发情况的适应能力较弱，预测精度仍有提升空间。

（2）功能实现深度有限

部分功能仅实现基础版本，例如排班推荐仍以规则方式为主，未实现完全自动化决策。

（3）系统规模较小

当前系统主要面向模拟环境，未在实际医院场景中部署，缺乏大规模数据验证。

（4）安全机制有待加强

虽然引入了 JWT 与 Redis，但在数据加密、防攻击等方面仍需进一步完善。

1.3 未来研究方向

查重 56%

针对上述不足，后续可以从以下几个方面进行改进与扩展：

（1）引入更复杂的预测模型

可尝试结合机器学习或深度学习方法（如 LSTM），提升预测精度，使结果更加稳定。

（2）优化排班推荐算法

在现有基础上引入多因素决策模型，例如结合医生工作量、患者偏好等因素，实现更加合理的排班方案。

（3）完善系统功能

进一步扩展系统功能，如增加在线支付、消息提醒、移动端支持等，提高系统实用性。

（4）提升系统安全性

在现有认证机制基础上，引入更完善的安全策略，如接口限流、数据加密传输等，增强系统抗风险能力。

（5）结合真实场景验证

未来可在实际医院或模拟真实数据环境中进行测试，以验证系统的实用价值。

参考文献

- [1] 夏大翔,张帧,张贝贝,等.基于微服务架构的电子病历智能内涵质控系统建设实践[J].中国卫生信息管理杂志,2025,22(05):695-701.
- [2] 林海祥.电子病历设计与应用——以厦门市仙岳医院为例[J].中国信息界,2024,(03):20-22.
- [3] 许辉,张志霞,方鹏骞.智慧化医院视角下公立医院门诊服务流程管理及其优化策略[J].中国医院,2024,28(11):85-89.DOI:10.19660/j.issn.1671-0592.2024.11.19.
- [4] 徐锡武,张静,蒋蕴雅,等.推行门诊电子病历的实践与思考[J].中国卫生质量管理,2021,28(06):10-13.DOI:10.13912/j.cnki.chqm.2021.28.6.03.
- [5] Bindu S , Ashok K , Krishnamurthy K ,et al.ClinicCare Manager: An Efficient Electronic Health Record (EHR) and Healthcare Management System Using Spring Boot and React[J].2024 International Conference on IoT Based Control Networks and Intelligent Systems (ICICNIS), 2024:1464-1470.DOI:10.1109/icicnis64247.2024.10823253.
- [6] 秦虎,时艳博,王帅同.基于结构化电子病历的医疗质量管理体系应用研究[J].中国数字医学,2020,15(02):13-14+17.
- [7] 毛戈,李晶,姚弘毅.基于智慧医院的电子病历应用和设计[J].湖北大学学报(自然科学版),2021,43(06):706-712.
- [8] 胡小勇.基于 SpringBoot 的医院门诊管理信息系统的设计与实现[D].华中科技大学,2021.DOI:10.27157/d.cnki.ghzku.2021.001118.
- [9] 冯尘尘,张欣莉,刘嘉怡,等.国内外门诊预约挂号调度系统研究进展[J].西南国防医药,2021,31(03):265-268.
- [10] 张强.我国智慧医院发展情况与趋势分析[J].中国医院建筑与装备,2022,23(08):50-54.
- [11] 李宝,路雅.基于微信小程序的预约挂号系统设计与实现[J].电子设计工程,2024,32(18):32-36.DOI:10.14022/j.issn1674-6236.2024.18.007.
- [12] 陈文娟,林建潮.基于季节 ARIMA 模型对某三级综合性医院门诊量的预测研究[J].中国医院统计,2024,31(03):185-188.
- [13] 王可欣,焦阳阳,吴子怡,等.公立医院智慧门诊一体化服务平台的探索与实践[J].中国医药导报,2023,20(22):162-166.DOI:10.20047/j.issn1673-7210.2023.22.36.
- [14] Choi JS, Lee JH, Park JH, Nam HS, Kwon H, Kim D, et al. Design and implementation of a

seamless and comprehensive integrated medical device interface system for outpatient electronic medical records in a general hospital[J]. International Journal of Medical Informatics, 2011, 80(4): 274-285. DOI: 10.1016/j.ijmedinf.2010.11.007.

- [15] Zhang Y, Li X, Wang J, et al. Predicting monthly hospital outpatient visits based on meteorological environmental factors using the ARIMA model[J]. Scientific Reports, 2023, 13: 29897. DOI:10.1038/s41598-023-29897-y.